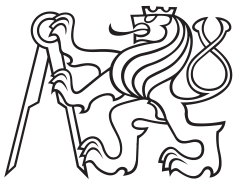
Master Thesis



Czech Technical University in Prague

F3

Faculty of Electrical Engineering

Numerical Optimal Control for Minimum Lap Time

Bc. Richard Ciglanský

Supervisor: doc. Ing. Zdeněk Hurák, Ph.D.
May 2026

FAKULTA ELEKTROTECHNICKÁ
FACULTY OF ELECTRICAL ENGINEERING
Technická 2
166 27 Praha 6

DECLARATION

I, the undersigned

Student's surname, given name(s):	Ciglanský Richard
Personal number:	498930
Programme name:	Cybernetics and Robotics

hereby declare that the Master's Thesis titled

Numerical Optimal Control for Minimum Lap Time

is my own independent work and that I have declared all sources of information used in accordance with the Methodological Guideline for adhering to ethical principles when elaborating an academic final thesis and the Framework Rules for the use of artificial intelligence at CTU for study and pedagogical purposes in Bachelor and continuing Masters studies.

I declare that I have used artificial intelligence tools during the preparation and writing of the final thesis.
I've verified the generated content. I confirm that I am aware that I am fully responsible for the content of the final thesis.

In Prague on 20.05.2026

Bc. Richard Ciglanský
.....
student's signature

Abstract

The thesis investigates and implements ways of solving the minimum-time maneuvering problem (lap-time simulation) for road vehicles. It divides the problem into smaller subproblems for which mature software methods already exist. The subproblems are: race-car modeling, transcription of the optimal control problem into an NLP, solving the NLP, solving sets of linear equations, and sensitivity analysis. A Julia-based implementation built on JuMP and Ipopt is presented. The framework supports several collocation schemes (Gauss-Legendre, Radau, and Lobatto) together with adaptive mesh refinement. Various vehicle models of increasing complexity, transcription strategies, and linear solver backends (MUMPS, HSL MA57/MA97) are benchmarked on representative circuits, and the parametric sensitivity of the optimal lap time is analyzed.

Keywords: NLP, optimization, automatic differentiation, twin-track model, Pacejka, Julia, scalability, Ipopt, MadNLP, MUMPS, MA97, CuDSS, sensitivity analysis

Supervisor: doc. Ing. Zdeněk Hurák, Ph.D.

Abstrakt

Táto práca skúma a implementuje metódy riešenia problému minimálneho času manévrovania (simulácia času na kolo) pre cestné vozidlá. Problém je rozdelený na menšie podproblémy, pre ktoré už existujú vyspelé softvérové nástroje. Týmito podproblémami sú: modelovanie pretekárskeho vozidla, transkripcia úlohy optimálneho riadenia na úlohu nelineárneho programovania (NLP), riešenie NLP, riešenie sústav lineárnych rovníc a citlivostná analýza. Každý podproblém je skúmaný v kontexte dynamiky vozidla a je predstavená implementácia v jazyku Julia postavená na knižnici JuMP. Implementácia podporuje viacero kolokačných schém (Gauss-Legendre, Radau a Lobatto) spolu s adaptívnym zjemňovaním siete. Rôzne modely vozidiel rastúcej zložitosti, stratégie transkripcie a backendy lineárnych riešičov (MUMPS, HSL MA57/MA97) sú porovnané na reprezentatívnych okruhoch a je analyzovaná parametrická citlivosť optimálneho času na kolo.

Klíčová slova: NLP, simulace času na kolo, optimální řízení, Ipopt, vozidlová dynamika, kolokační metody, Julia, citlivostní analýza

Překlad názvu: Metody numerického optimálního řízení pro minimalizaci času na kolo

Contents

Figures

Chapter 1
Introduction

1.1 Motivation

The topic of this thesis is motivated by my work in Formula Student team eForce Prague Formula, where I worked on real-time vehicle dynamics control software. There I got directly exposed to problems faced by racing teams: should we increase the aerodynamic downforce at the cost of higher drag? Should we put the accumulator in the middle of the vehicle to decrease the moment of inertia? Should the aerodynamics be designed for straight-line drive, or for steady-state turning? How can the limited energy from the accumulator be used more effectively, and how much faster can we go with a better power-limiting algorithm? How large should the accumulator be? Which torque allocation algorithm should be used? How to design the suspension kinematics to maximize the tire utilization? Is rear-wheel steering worth the increased weight? All of these problems can be analyzed using a purpose-built tool, good engineering practices, or pen and paper with a heavily simplified model. The Formula Student season is 10 months for development and construction of the vehicle, and 2 months for competition. That puts enormous pressure on deciding on a vehicle concept *fast*. In my opinion, what Formula Student teams need is an open-source tool to quickly evaluate different car concepts. In my bachelor thesis I modified existing code into a prototype of such a tool. The goals of this thesis come from computational and user-experience lessons learned from my previous tool.

1.2 Goals

- The primary goal of this thesis is to investigate available methods for numerical optimal control with respect to their suitability for planning an optimal trajectory of a vehicle aiming to achieve the minimum lap time on a racing circuit.
- Formulate the minimum lap-time problem as an optimal control problem.
- Conduct a brief literature review of fundamental methods for numerical optimal control, focusing primarily on direct transcription methods.

- Implement and benchmark the performance of the available numerical methods.
- The implementation should support scalability across vehicle models, transcription schemes, and solvers.
- Make the code and thesis available on GitHub: `https://github.com/ceserik/SLapSim.jl`.

1.3 Structure of thesis

The first part of the thesis is vehicle modeling in the context of optimal control. A universal vehicle and track model structure inspired by graph theory from [16] is presented. Following this, various methods of transcribing the problem into nonlinear programming or solving the problem are introduced. The subsequent section describes the basic working principles of Lagrange-Newton solvers of nonlinear programming. After that, frameworks for creating and solving nonlinear programming problems are reviewed. Finally, my implementation using one framework with a benchmark of different vehicle models, transcription methods and solvers follows.

1.4 Survey of lap-time simulators

Lap-time simulation dates back to the 1950s [53]. Computers became common in the 1970s for lap-time simulation, and quasi-steady-state (QSS) methods became the standard approach for minimum lap-time simulation. QSS is still what most race engineers mean by "lap-time simulation". In QSS the vehicle is a point mass on a fixed racing line, with lateral and longitudinal accelerations bounded by a steady-state performance envelope [63]. Transient effects such as yaw acceleration are ignored. Minimum lap-time simulation was first formulated as an optimal control problem in the late 1980s [35]. A survey of current methods is given in [53]. The work of Perantoni and Limebeer [60] is the main reference: it uses a track-fixed coordinate system, switches the independent variable from time to traveled distance, and applies the formulation to a twin-track Formula 1 model with some vehicle parameters as decision variables. Parts of this thesis build on it. Minimum lap-time simulation is still an active research area because better predictions give measurable setup gains, but higher fidelity models cost more solver time. The authors of [2] use sequential convex programming to cut compute cost without losing accuracy.

A very popular lap-time simulator is OptimumLap from OptimumG [3]. It is a free tool advertised to achieve 10% accuracy with real vehicles. It uses Quasi-Steady-State methods (QSS) and is the standard tool for lap-time simulations. An open-source implementation of the same method is [54]. My implementation of this method is described in Chapter 6. QSS methods are

very fast, but the underlying models are very simple and unable to describe more complex vehicle behavior.

A popular open-source C++ implementation of a lap-time simulator using nonlinear programming and automatic differentiation is [52]. It uses IPOPT and CppAD [7]. The vehicle model is a 6DOF twin-track. Other implementations include work from Austria and Germany [37] and a class project from the Georgia Institute of Technology [1].

Part I

Theory

Chapter 2

Vehicle and Track modeling framework

The goal of the thesis is to select a sufficiently accurate yet computationally tractable model of the vehicle. There is no single vehicle model that fits all requirements of race engineers. For each problem, a different complexity of the vehicle model is sufficient, e.g. for analysis of a power-limit algorithm, the lateral dynamics of the vehicle can be heavily simplified, and simple transcription and solving methods yield sufficient accuracy within a fraction of a second. This is described in Section 3.1.1. Whereas analysis of the torque allocation problem requires the full twin-track model and more computationally expensive transcription and solving methods. Therefore, a vehicle modeling framework is needed to satisfy the needs of different types of analyses.

2.1 Graph-centered approach for vehicle modeling

The structure of the vehicle model has to allow extending or changing components, so that different vehicle concepts can be simulated with minimal coding. The book [16] describes a modeling framework that allows creating car components as modules with a flow and effort interface. This common interface heavily simplifies the process of creating a car model, as modules can be reused and chained together easily. The vehicle components are: chassis, wheel pivot, tire, aerodynamics, suspension, motor, and accumulator.

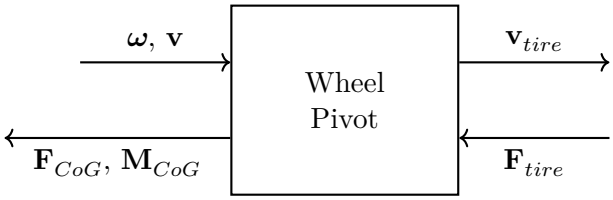


Figure 2.1: Wheel Pivot

2.2 Tire-road interface modeling

The role of the tire model in this thesis is to calculate forces acting on the tire from the relative velocity and angle between tire and road. The input of

a general tire model is 3 velocities and 3 angles, and the output is 3 forces.

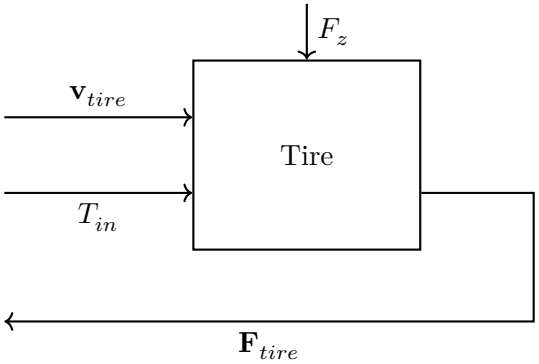


Figure 2.2: Tire component

Most forces that govern vehicle movement are transferred by the tires. The tire is considered to be the most important system of a vehicle and it is notoriously difficult to model [55]. Thus tires need to be modeled carefully. Not only are there many parameters, such as tire wear, tire temperature, road roughness, road wetness, tire pressure, etc., that affect the resultant forces, but they are also constantly changing during the lap. Therefore they are difficult to measure and model precisely. This paragraph is adapted from my bachelor thesis [19]. Implementation of specific models is covered in Part III, Implementation.

2.2.1 Linear tire

This tire model maps lateral slip angle

$$\alpha = -\arctan\left(\frac{v_y}{v_x}\right), \tag{2.1}$$

into lateral force

$$F_y = \frac{\alpha}{\alpha_{\max}} \mu F_z. \tag{2.2}$$

The tire model does not consider spinning of the wheels. Therefore, there is no longitudinal slip, and the motor force acts directly on the wheel pivot point.

$$F_x = \frac{T_{\text{wheel}}}{R} + F_{\text{brake}}. \tag{2.3}$$

Forces are constrained by friction ellipse

$$F_x^2 + F_y^2 \leq (\mu F_z)^2. \tag{2.4}$$

The implication of neglecting tire spinning is that energy from the motor is not used to spin up the wheels and goes entirely into accelerating the car, creating a large discrepancy from how a real vehicle operates. In this and the following models this effect is not accounted for.

2.2.2 Magic Formula tire

The Magic Formula tire model, developed by Hans B. Pacejka [36], is an empirically derived formula that describes tire forces. The full Magic Formula model has around 80 parameters and many variants. The variant described here was taken from [1] and has only 6 parameters.

$$\alpha = -\arctan\left(\frac{v_y}{v_x}\right). \tag{2.5}$$

Parameter D is the peak factor

$$D = \mu F_z \left(1 - \epsilon \frac{F_z}{F_{z,\text{ref}}}\right), \tag{2.6}$$

ϵ models degressive behavior, the peak lateral force of a real tire does not grow linearly with normal force. $F_{z,\text{ref}}$ is the nominal normal tire load [1]. The lateral force is then computed according to the Magic Formula from [36],

$$F_y = D \sin\Big(C \arctan(B\alpha - E\,(B\alpha - \arctan(B\alpha)))\Big), \tag{2.7}$$

where C is the shape factor, B is the stiffness factor and E is the curvature factor. These parameters are fitted to real tire measurement data. Then tire rolling resistance C_r is added and the friction ellipse is the same as for the linear tire,

$$F_x = \frac{T_{\text{wheel}}}{R} + F_{\text{brake}} + C_r F_z, \tag{2.8}$$

$$\left(\frac{F_x}{\mu F_z}\right)^2 + \left(\frac{F_y}{\mu F_z}\right)^2 \leq 1. \tag{2.9}$$

2.3 Chassis model

The chassis is modeled as a point mass located at the center of gravity (CoG), carrying the full vehicle mass and yaw inertia, with the wheel pivot points defined relative to it. The chassis model is responsible for computing the state-derivative vector from all forces and moments acting on the CoG. Equations 2.10–2.13 describe the computation.

Table 2.1: Parameters of the chassis model.

Symbol	Unit	Description
m	kg	Mass
I_z	$kg \cdot m^2$	Yaw Inertia
w	m	Width of vehicle

$$\mathbf{M}_{CoG,i} = \mathbf{r}_i \times \mathbf{F}_i, \qquad \mathbf{F}_i = \begin{bmatrix} F_{x_i} \\ F_{y_i} \\ 0 \end{bmatrix}. \tag{2.10}$$

$$\mathbf{F}_{CoG} = \sum_i \mathbf{F}_i + \begin{bmatrix} F_{\text{drag}} \\ 0 \\ 0 \end{bmatrix}, \qquad \mathbf{M}_{CoG} = \sum_i \mathbf{M}_{CoG,i}. \tag{2.11}$$

$$\dot{\mathbf{v}} = \frac{\mathbf{F}_{CoG}}{m} - \boldsymbol{\omega} \times \mathbf{v}, \qquad \dot{\boldsymbol{\omega}} = \frac{\mathbf{M}_{CoG}}{I}. \tag{2.12}$$

$$\mathbf{x} = \begin{bmatrix} v_x \\ v_y \\ \psi \\ \dot{\psi} \end{bmatrix}, \qquad \dot{\mathbf{x}} = \begin{bmatrix} \dot{v}_x \\ \dot{v}_y \\ \dot{\psi} \\ \ddot{\psi} \end{bmatrix}. \tag{2.13}$$

The chassis also imposes a constraint on the location on the track to prevent the vehicle from leaving it.

2.4 Wheel Pivot model

The wheel pivot point is modeled as a transformer. It defines the transformation of CoG velocities into the tire frame and then transforms tire forces and torque into the pivot frame. The angular velocity and position vectors are formed from the yaw rate $\dot{\psi}$, the longitudinal distance l_i of the axle from the center of gravity, and the lateral distance $\pm b/2$ of the wheel from the vehicle centerline:

$$\boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ \dot{\psi} \end{bmatrix}, \qquad \mathbf{r}_i = \begin{bmatrix} l_i \\ b/2 \\ 0 \end{bmatrix}, \tag{2.14}$$

where b is the track width. Velocity at each tire pivot point is:

$$\mathbf{v}_{p_i} = \mathbf{v} + \boldsymbol{\omega} \times \mathbf{r}_i. \tag{2.15}$$

Velocities are then rotated from the pivot frame into the tire frame using the steering angle δ_i:

$$R_t = \begin{bmatrix} \cos\delta_i & -\sin\delta_i & 0 \\ \sin\delta_i & \cos\delta_i & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{v}_{t_i} = R_t\, \mathbf{v}_{p_i}. \tag{2.16}$$

Tire forces F_{tx} and F_{ty} are computed from $\mathbf{v}_{t_i}$ by the tire model. Forces are then rotated back to the pivot frame:

$$\begin{bmatrix} F_{x_i} \\ F_{y_i} \end{bmatrix} = \begin{bmatrix} \cos\delta_i & \sin\delta_i \\ -\sin\delta_i & \cos\delta_i \end{bmatrix} \begin{bmatrix} F_{tx} \\ F_{ty} \end{bmatrix}. \tag{2.17}$$

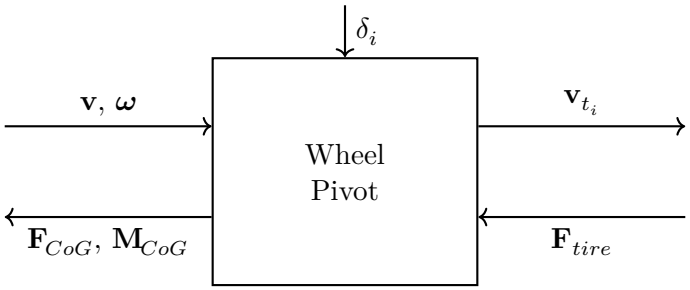


Figure 2.3: Wheel Pivot component

2.5 Aerodynamics model

The aerodynamic model is treated as a nonlinear resistance element, mapping velocity into forces. Drag and down-force scale with the square of longitudinal velocity:

$$F_{\text{drag}} = \tfrac{1}{2}\,\rho\,C_D\,v_x^2, \qquad F_{\text{lift}} = \tfrac{1}{2}\,\rho\,C_L\,v_x^2. \tag{2.18}$$

The down-force is split between front and rear axles by the center of pressure ratio CoP $\in [0, 1]$, where CoP $= 0.5$ corresponds to a balanced distribution. Air density ρ is taken as $1.225\,\text{kg/m}^3$ at sea level. Coefficients C_L, C_D and CoP used per vehicle model are listed in Table 2.2.

Table 2.2: Parameters of the aerodynamic model.

Symbol	Unit	Description
C_L	—	Lift coefficient (negative = down-force)
C_D	—	Drag coefficient
CoP	—	Center of pressure (front axle share, 0–1)
ρ	kg/m^3	Air density

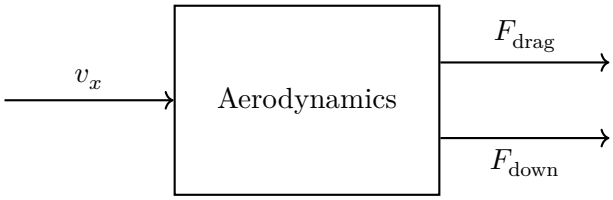


Figure 2.4: Aerodynamics component

This model is sufficient when the vehicle is driving relatively straight, i.e. when lateral acceleration is roughly zero. With high lateral acceleration this model is no longer valid, because air speeds differ on the left and right side of the car and the car also starts rolling, one side of the vehicle is closer to the ground than the other, creating an imbalance of lift forces.

2.6 Suspension model

The purpose of the suspension in a car is to maintain tire contact, distribute the load from lateral load transfer to better utilize the tires, and use kinematics to make use of camber thrust. Commonly the suspension of a car consists of springs, dampers and the mounting with tie rods. In this thesis, the suspension model had to be significantly simplified. Modeling each wheel with a damper, spring and its own kinematics is beyond the scope of this thesis. However, the model structure allows for it, but I did not have enough time to implement it. Such a model would be very beneficial toward the goal of scalability. Without it, the model scalability is limited, as dividing force between more than two wheels on an axle is a statically indeterminate problem.

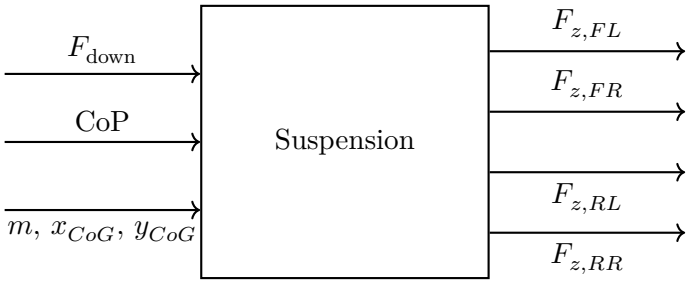


Figure 2.5: Suspension component

Two suspension variants are used in this thesis: static distribution and dynamic distribution.

2.6.1 Static suspension

The static suspension distributes the vehicle's mass and the force from aerodynamics according to the position of the center of gravity and the center of pressure. This trivial implementation is possible only for two axles, three or more axles would make this a statically indeterminate system.

$$\begin{aligned} F_{z,FL} &= (mg\,x_{\text{CoG}} + F_{\text{lift}}\,\text{CoP})(1 - y_{\text{CoG}}) \\ F_{z,FR} &= (mg\,x_{\text{CoG}} + F_{\text{lift}}\,\text{CoP})\,y_{\text{CoG}} \\ F_{z,RL} &= (mg\,(1 - x_{\text{CoG}}) + F_{\text{lift}}(1 - \text{CoP}))(1 - y_{\text{CoG}}) \\ F_{z,RR} &= (mg\,(1 - x_{\text{CoG}}) + F_{\text{lift}}(1 - \text{CoP}))\,y_{\text{CoG}} \end{aligned} \tag{2.19}$$

2.6.2 Dynamic suspension

This suspension variant takes into account the load transfer caused by longitudinal and lateral accelerations. It is adapted from [1].

$$\begin{aligned}
F_{z,FL} &= \left(mg\,x_{\mathrm{CoG}} - \frac{m\,a_x h_{\mathrm{CoG}}}{L}\right)(1 - y_{\mathrm{CoG}}) + \tfrac{1}{2}F_{\mathrm{down}}\,\mathrm{CoP} - \tfrac{1}{2}\Delta F_y \\
F_{z,FR} &= \left(mg\,x_{\mathrm{CoG}} - \frac{m\,a_x h_{\mathrm{CoG}}}{L}\right) y_{\mathrm{CoG}} + \tfrac{1}{2}F_{\mathrm{down}}\,\mathrm{CoP} + \tfrac{1}{2}\Delta F_y \\
F_{z,RL} &= \left(mg\,(1 - x_{\mathrm{CoG}}) + \frac{m\,a_x h_{\mathrm{CoG}}}{L}\right)(1 - y_{\mathrm{CoG}}) + \tfrac{1}{2}F_{\mathrm{down}}\,(1 - \mathrm{CoP}) - \tfrac{1}{2}\Delta F_y \\
F_{z,RR} &= \left(mg\,(1 - x_{\mathrm{CoG}}) + \frac{m\,a_x h_{\mathrm{CoG}}}{L}\right) y_{\mathrm{CoG}} + \tfrac{1}{2}F_{\mathrm{down}}\,(1 - \mathrm{CoP}) + \tfrac{1}{2}\Delta F_y
\end{aligned} \tag{2.20}$$

ma_x is the longitudinal force, estimated as the sum of driving, braking, drag and rolling-resistance forces:

$$ma_x \approx F_{\mathrm{drive}} + F_{\mathrm{brake}} - F_{\mathrm{drag}} - C_r mg. \tag{2.21}$$

ΔF_y is a new decision variable that describes lateral load transfer. It is subject to the constraint

$$\Delta F_y = \frac{h_{\mathrm{CoG}}}{t}\sum_{i=1}^{4} F_{y,i}, \tag{2.22}$$

where $F_{y,i}$ are the lateral forces at the pivot points of the wheels. This is a quasi-steady-state model of suspension, as the load transfer is considered to be instant. In a real vehicle, a system of spring and damper on each wheel introduces more complex dynamics.

2.7 Gearbox model

The gearbox is a textbook example of a transformer and a resistive element.

$$T_{\mathrm{out}} = \eta\, i\, T_{\mathrm{in}}, \qquad \omega_{\mathrm{out}} = \frac{\omega_{\mathrm{in}}}{i}. \tag{2.23}$$

Table 2.3: Parameters of the gearbox model.

Symbol	Unit	Description
i	—	Gear ratio (input to output speed)
η	—	Mechanical efficiency

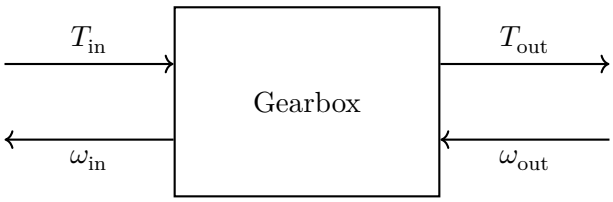


Figure 2.6: Gearbox component

2.8 Accumulator model

The accumulator model sums power from the motors

$$P_{\text{acc}}(t) = \sum_{i=1}^{N_m} P_{\text{el},i}(t), \tag{2.24}$$

and then constrains the power according to the accumulator limits

$$P_{\text{acc}}^{\min} \leq P_{\text{acc}}(t) \leq P_{\text{acc}}^{\max}. \tag{2.25}$$

What is missing in this thesis is a total energy constraint for a lap.

2.9 Motor model

The motor is modeled as an effort source with efficiency. The efficiency is traditionally modeled using a *max* function (Equation 2.26), but this function is not differentiable.

$$P_{\text{mech}} = T\,\omega, \qquad P_{\text{elec}} = \max\left(\frac{P_{\text{mech}}}{\eta},\ P_{\text{mech}}\,\eta\right). \tag{2.26}$$

The max function cannot be used in nonlinear programming, so a different formulation has to be used. This thesis uses the constraints

$$\begin{aligned} P_{\text{elec}} &\geq \frac{P_{\text{mech}}}{\eta}, \\ P_{\text{elec}} &\geq P_{\text{mech}}\,\eta. \end{aligned} \tag{2.27}$$

A problem with this formulation is that when full power is not required, i.e. when P_{acc} is strictly inside the constraints

$$P_{\text{acc}}^{\min} \leq P_{\text{acc}}(t) \leq P_{\text{acc}}^{\max}, \tag{2.28}$$

the power draw from the accumulator may be higher compared to the classical formulation with the *max* function. Another possible formulation is a smooth max approximation using arctan, $\sqrt{\cdot}$, or the softmax function. I have covered this approach in my bachelor thesis [19].

Table 2.4: Parameters of the motor model.

Symbol	Unit	Description
T	Nm	Shaft torque
η	—	Motor + inverter efficiency
P_{mech}	W	Mechanical shaft power
P_{elec}	W	Electrical port power (battery side)

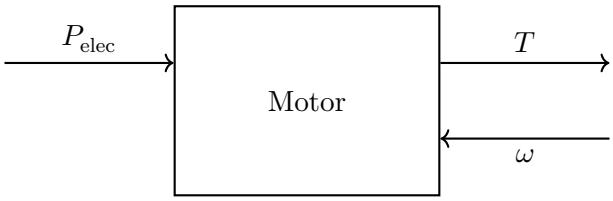


Figure 2.7: Motor component

2.10 Brake model

Braking force is distributed among the wheels by the brake bias ratio. To exclude simultaneous braking and accelerating, a complementarity constraint is used:

$$T_{\text{m}} \cdot F_{\text{b}} \leq \varepsilon, \quad T_{\text{m}} \geq 0, \; F_{\text{b}} \geq 0, \tag{2.29}$$

where T_{m} is the driving torque (or the sum of driving torques) and F_{b} is the braking force. A cleaner alternative is to penalize energy consumption in the objective, then the explicit complementarity constraint is not required.

2.11 Car models

These models must work when used in NLP transcription for solving the boundary value problem, but they also must work for the initial value problem, when the model is initialized or verified against transcription error. Some toolboxes can automatically create constraints from ODE models [76]. In my implementation I have an if statement that switches between the IVP and BVP variants of the model equations. A major disadvantage of this approach is that the equations for both variants must be effectively the same, and finding bugs is difficult.

2.11.1 Generic car model structure

Leveraging the full potential of automatic differentiation, any car parameter can be assigned as a state, control, static parameter, decision variable, or sensitivity-analysis variable. This allows changing a fixed parameter into a degree of freedom and deciding which parameters should be controlled. It also allows finding the optimal vehicle parameters, such as brake bias, aerodynamic balance, and gear ratio, during lap-time optimization. This is

shown in Figure 2.8. When a parameter is chosen as a sensitivity-analysis variable, the impact of changing it is cheaply computed after the optimization is finished, see Section 3.7. Each parameter holds three values: lower bound, upper bound, and nominal value. The bounds are passed directly to the NLP as variable limits and are reused to scale the variable. The constraint that wheels cannot leave the track is implemented using upper and lower bounds on the distance from the centerline, which is a state of the car.

All vehicles must have at least these states:

- v_x — longitudinal velocity in the body frame [m/s],
- v_y — lateral velocity in the body frame [m/s],
- ψ — heading angle relative to the track tangent [rad],
- $\dot{\psi}$ — yaw rate [rad/s],
- n — signed lateral distance from the centerline [m],
- t — elapsed lap time [s].

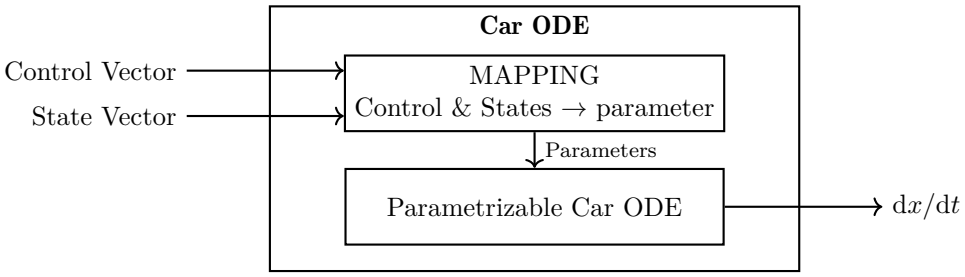


Figure 2.8: Generic car model structure

2.11.2 Mass-point quasi-steady-state model

A simple and useful car model can be built using these equations

$$F_z = \tfrac{1}{2}\,\rho\,C_L\,v_x^2, \tag{2.30}$$

$$F_y = m\,v_x^2\,c, \tag{2.31}$$

$$F_{x,\mathrm{pwr}} = \frac{P_{\max}}{v_x}, \tag{2.32}$$

$$F_{x,\max}^2 = \max\Big(\left[(F_z + mg)\mu\right]^2 - F_y^2,\ 0\Big), \tag{2.33}$$

$$F_x^2 + F_y^2 \le \left[(F_z + mg)\,\mu\right]^2, \tag{2.34}$$

$$|F_x| \le \min\big(F_{\mathrm{mot,max}},\ F_{x,\mathrm{pwr}}\big), \tag{2.35}$$

$$\dot{v}_x = \frac{F_x - \tfrac{1}{2}\,\rho\,C_D\,v_x^2}{m}. \tag{2.36}$$

Here ρ is the air density, C_L is the lift coefficient, v_x is the longitudinal speed, m is the mass, c is the curvature of the track at the current point, $P_{\max}$ is

the maximum power of the motor, μ is the friction coefficient, and C_D is the drag coefficient. This model has only one state, v_x, which makes it possible to use specialized lap-time simulation solvers. Variations of this model, where for example $P_{\max}$ is a lookup table or the grip does not scale linearly with down-force, are commonly used in motorsport.

2.11.3 Single-track

A more complex and useful model is the single-track model. In this model, suspension effects are neglected. The single-track model is still considered a simplified model and is used most commonly when more complex models are too computationally expensive. A computational graph of how this model is assembled from the defined components is shown in Figure 2.9.

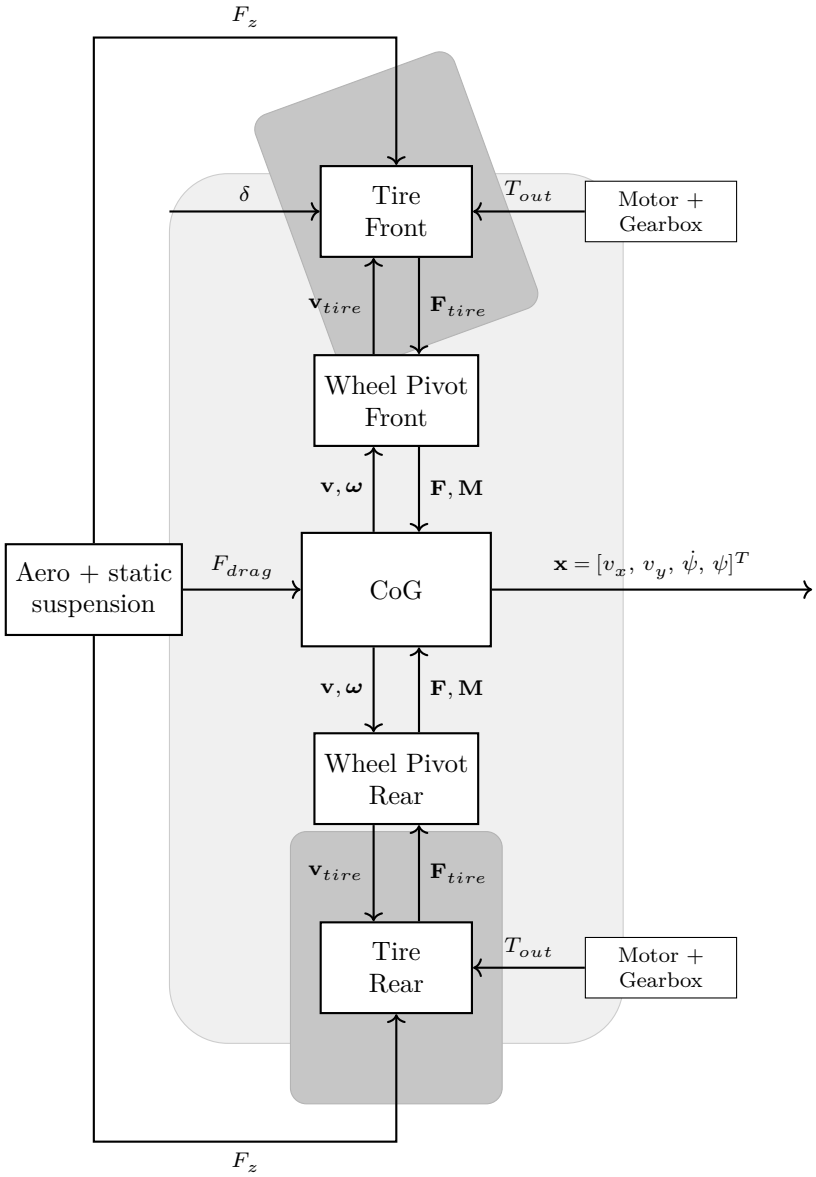


Figure 2.9: Single-track (bicycle) model

2.11.4 Twin-track model

The twin-track model treats all four tires separately. This captures the left-right load transfer. The computational graph is shown in Figure 2.10.

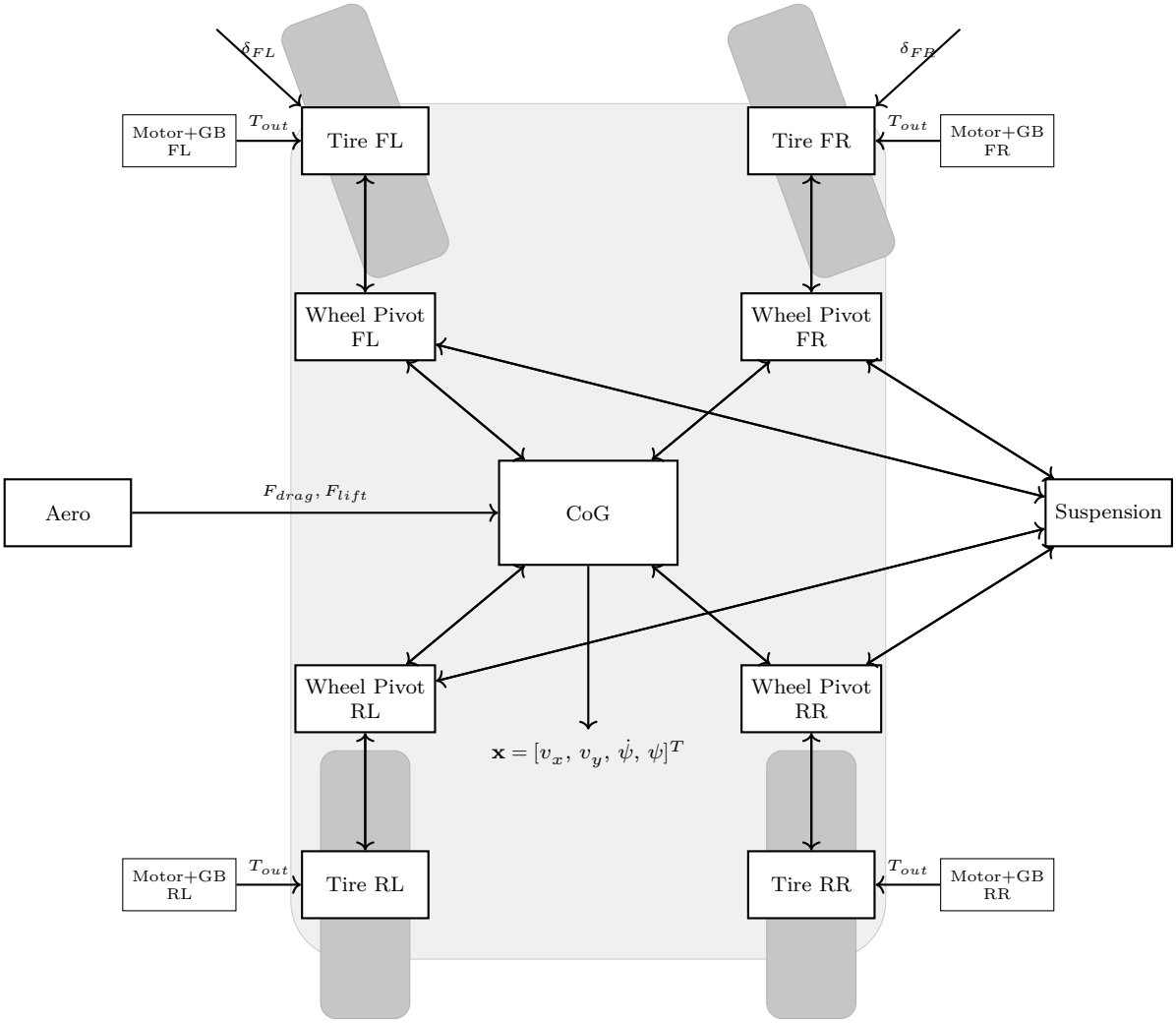


Figure 2.10: Twin-track model

2.12 Track modeling

The track is sampled at discrete arc length nodes s_i and stored as a set of fields evaluated at each node:

- centerline coordinates $x(s)$, $y(s)$,
- heading $\theta(s)$,
- curvature $\kappa(s)$,
- left and right track widths $w_L(s)$, $w_R(s)$,
- air density $\rho(s)$,
- friction coefficient $\mu(s)$,

- inclination $\phi(s)$.

Each field is linearly interpolated between the segments. Obtaining a suitable track for optimization from GPS is non-trivial. Simple methods lead to noisy data, which then corrupts the optimization result. Therefore, the smoothing method described in [60] has been used.

2.13 Change of independent variable, time to path

When the differential equations for vehicle movement are stated, it is beneficial in lap time simulation to describe the car states not in time but in distance travelled. This introduces a connection with the location on the track and reduces the number of state variables by one. This transformation is taken from [60].

$$\varepsilon = \psi - \theta(s) \tag{2.37}$$

where ψ is the car heading, and $\theta(s)$ is the heading of the track at distance s.

$$S_f = \frac{\mathrm{d}t}{\mathrm{d}s} = \frac{1 - n\,C(s)}{v_x \cos\varepsilon - v_y \sin\varepsilon}, \tag{2.38}$$

n is the vehicle's distance from the centerline, and $C(s)$ is the curvature of the track at distance s. The change in n is defined as:

$$\frac{\mathrm{d}n}{\mathrm{d}t} = v_x \sin\varepsilon + v_y \cos\varepsilon. \tag{2.39}$$

Then the model can be transformed from time as the independent variable to distance travelled along the centerline s as the new independent variable using the equation:

$$\frac{\mathrm{d}}{\mathrm{d}s}\begin{pmatrix} v_x \\ v_y \\ \psi \\ \dot{\psi} \\ n \\ t \end{pmatrix} = S_f \begin{pmatrix} \dot{v}_x \\ \dot{v}_y \\ \dot{\psi} \\ \ddot{\psi} \\ \dot{n} \\ 1 \end{pmatrix}. \tag{2.40}$$

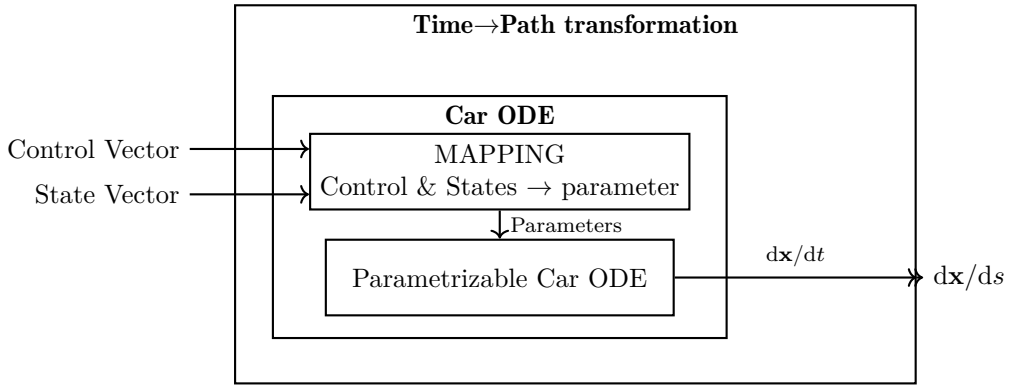


Figure 2.11: Car with a time-to-path transformation layer

Now the vehicle is modeled and placed on the track. The next step is to drive it as fast as possible.

Chapter 3

The Optimal Control Problem

In the previous chapter I described how to build the differential equations that govern the movement of a vehicle. This chapter describes two approaches to driving that vehicle optimally. First is a classical initial value problem or time-marching formulation, then a reformulation as a boundary value problem.

3.1 Minimum maneuvering time problem as an initial value problem

The most common method for solving an ODE is to use an initial value problem. An industry-standard high-fidelity multibody time-marching solver for vehicles is Car Maker from IPG [39]. It is great for driving simulation, testing, and development of advanced driver assistant systems. It can model vehicles and racetracks and can be used for lap time simulation, but with caveats. In time-marching solvers, a separate controller is required, which becomes part of the ODE. This approach gives a solution to the problem of how to control the car so that it completes a lap around a circuit. However, this solution is almost certainly not optimal for time or any meaningful objective. Another issue is that if the vehicle's parameters change and the controller does not, the resulting time can be exactly the same, because the controller cannot take advantage of different vehicle properties. Sensitivity analysis and lap time can be obtained, but it is skewed by the chosen controller. The controller can be tuned for a specific vehicle, but that will still not guarantee optimal control. Using time marching is not the best approach for sensitivity analysis and lap time simulation of a complex model.

When the vehicle model is simplified to a quasi-steady-state model, time-marching simulation can be used.

3.1.1 Quasi-steady-state lap time simulation

The following section describes a commonly used algorithm that uses only initial value solvers to compute the minimum maneuvering time. For the quasi-steady-state model 2.36, it is possible to compute an optimal speed profile using an algorithm well known among racing engineers. I also solved

the equivalent NLP and obtained the same result, so at least for this track it appears to be optimal. Figure 3.2 shows the speed profile obtained from this algorithm on the track from Figure 3.1. The start is the dot and the cross marks the finish.

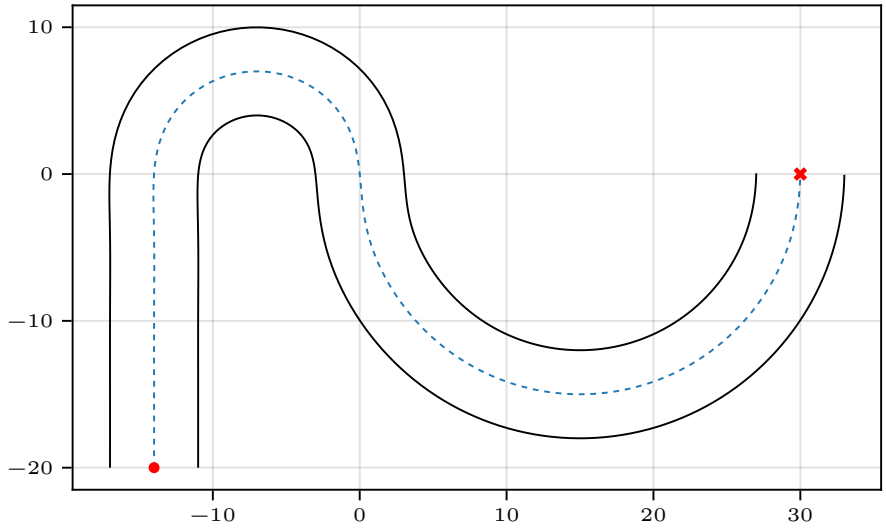


Figure 3.1: Test track used for the forward/backward pass example.

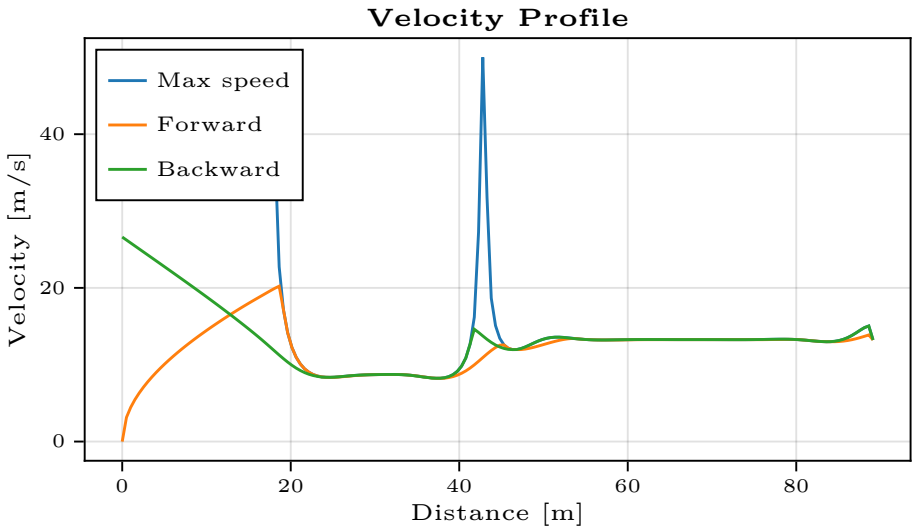


Figure 3.2: Speed profile from forward and backward passes with a quasi-steady-state model

The algorithm works in the following manner:

1. Maximum speed pass: From the vehicle model and curvature along the track, compute the maximum speed, the blue line in Figure 3.2.

2. Forward pass: Consider only the forward acceleration limits of the vehicle, with braking force unlimited. Run the vehicle from start to finish along the trajectory, applying maximum longitudinal acceleration and respecting the max speed limit. This is the orange line in Figure 3.2.

3. Backward pass: Consider only the braking acceleration limits of the vehicle, with acceleration force unlimited. Run the vehicle *backward, from finish to start* along the trajectory, applying maximum braking acceleration (braking now increases speed as the simulation is flipped) and respecting the max speed limit.

4. Take the minimum speeds of the forward and backward pass, which gives the minimum-time speed profile.

More complete description of this algorithm is in [18]. This is the classic approach to the minimum maneuvering time problem. The issue is that the trajectory of the vehicle on the track is fixed and cannot be optimized. The vehicle model must be heavily simplified. It does not work with integral constraints like accumulator energy. However, the computation time can be a fraction of a second for the entire lap. Tools using this method have proven to be extremely useful, and many different analyses can be performed by leveraging its computational speed. For example, by varying the friction coefficient at each corner, a corner's importance on lap time can be calculated [57].

3.2 Transcription methods

The previous section described automotive-specific methods of solving the minimum maneuvering time problem. This section explains methods of transcribing the problem into the language of optimal control problems. Transcription methods rewrite continuous (infinite-dimensional) OCP into finite-dimensional NLP. That means representing the continuous trajectory of a state by values sampled at discrete points. The terminology of transcription methods can be difficult to understand, as it is a complex topic and often articles present conflicting information [49]. The website [49] and other work from Matthew Kelly [46, 44, 47] offer a very digestible introduction to the topic. This thesis focuses only on direct methods. The classification of methods in this thesis is based on [49]. Figure 3.3 shows how transcription methods are commonly classified.

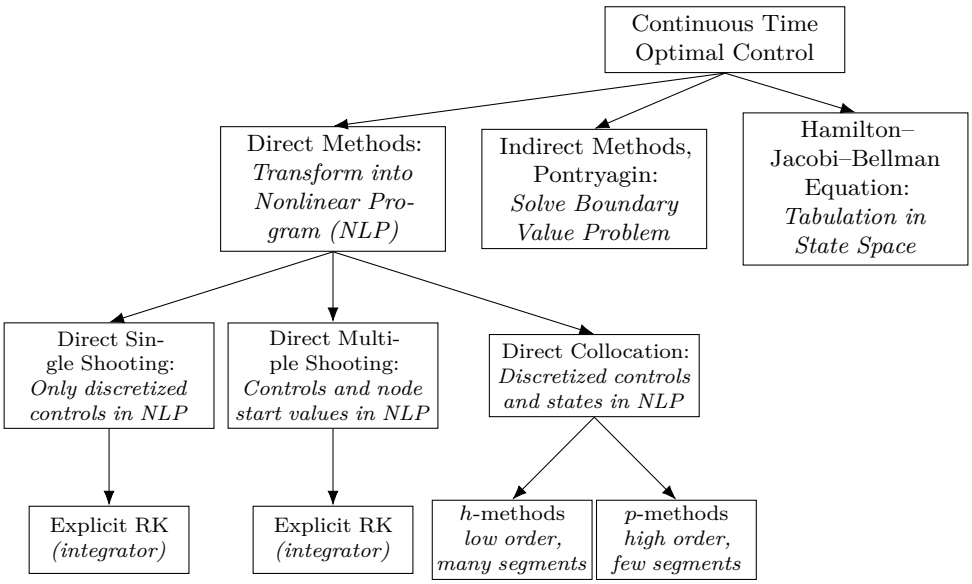


Figure 3.3: Tree of approaches to continuous-time optimal control, adapted from [33]

3.2.1 Shooting methods

Shooting methods get their name from an artillery cannon analogy. With a given powder load and target location, the aiming angle is adjusted to hit the target. An in-depth explanation with animation and example code is here [45]. This is a classical boundary value problem. Methods shoot, miss, adjust aim, shoot again, and iterate until the target is hit. These methods solve a boundary value problem using an IVP solver. More precisely,

1. Guess the controls
2. Compute the solution using an initial value problem solver
3. Compare the simulated value at the end of the interval from step 2 with the boundary value (target location)
4. Based on the error, update the controls and return to step 2

Shooting methods are split into single and multiple shooting. They differ in the way they handle state discretization. Single shooting discretizes the state only at the start and end of the interval and computes the trajectory in one shot. Multiple shooting discretizes the states along the trajectory and computes multiple shots, which can also be parallelized on a CPU [28]. Shooting methods are being progressively displaced by collocation methods [62].

3.2.2 Nonlinear Programming with Runge Kutta constraints

These methods do not use initial value solvers to solve a boundary value problem. Instead they use nonlinear programming. The most straightforward

method to transcribe a continuous optimal control problem into a nonlinear program is to use explicit Runge Kutta integration from the initial value problem as equality constraints on the dynamics of a vehicle. Using the Euler method, the constraints for the dynamics of the vehicle are:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + h \cdot f(\mathbf{x}_k, \mathbf{u}_k), \tag{3.1}$$

where f denotes the vehicle dynamics, h the step size, and $\mathbf{x}$ the state vector. This works, but it may be the worst possible transcription method because of the high error and instability of the Euler method [4]. Problems with the Euler method encountered when solving initial value problems also translate to nonlinear programming. Implicit RK methods can be used too. In initial value problems they are used less often, as they require iterative solving. An optimal control problem is a boundary value problem and not an initial value problem, therefore:

> …for a boundary value problem …any scheme becomes effectively implicit. Thus, the distinction between explicit and implicit initial value schemes becomes less important in the BVP context.

[12, p. 69]

For a boundary value problem it makes no sense to use explicit Runge Kutta methods, as they have inferior stability compared to implicit Runge Kutta methods.[62, p. 4]. Implicit Runge Kutta methods that are commonly used are from the LobattoIIIA family, but other methods can be used. The book [15] uses only these.

3.2.3 Collocation methods

> The concept of collocation is old and universal in numerical analysis …For ordinary differential equations it consists in searching for a polynomial of degree s whose derivative coincides ("co-locates") at s given points with the vector field of the differential equation …[5, p. 224]

The article [46] is an introductory explanation of the inner working and implementation of direct collocation methods. The theory here is based on it and on [29]. More in-depth theory behind these methods was developed by Lloyd N. Trefethen in [70] and the implementation in [71]. Collocation methods are a subset of implicit Runge-Kutta methods. The distinguishing property is that the solution of a boundary value problem is approximated by a piecewise polynomial in the independent variable (distance along the centerline). On each interval the polynomial is fixed by requiring it to satisfy the ODE exactly at N chosen points called the collocation nodes.

There are numerous ways to construct a collocation method. In optimal control the classic methods are those based on roots of Legendre or Chebyshev type I polynomials, Lagrange polynomials [25, 30], and the Gauss quadrature rule. In this thesis only Legendre and Gauss quadrature based collocation

methods are covered. Figure 3.4 depicts multiple ways of constructing collocation methods.

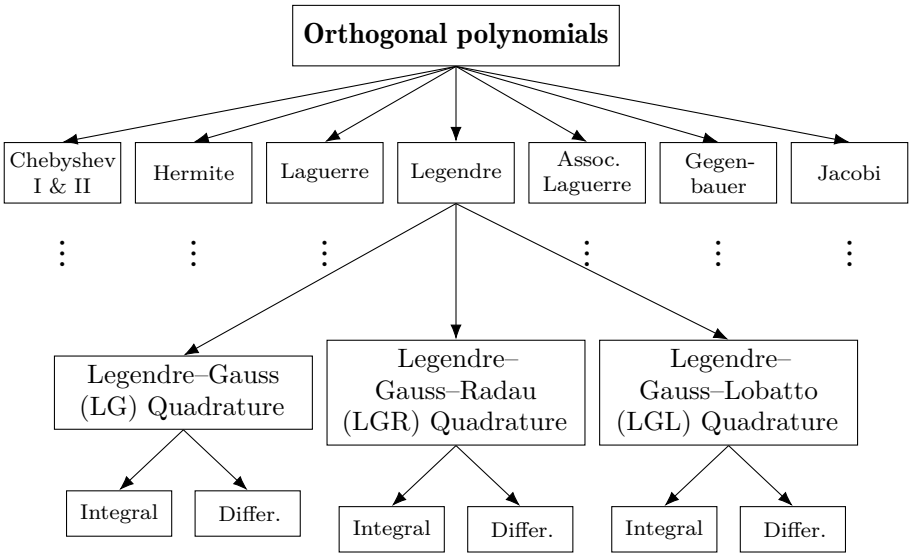


Figure 3.4: Collocation transcription methods

Lagrange polynomial

The first part of creating a collocation method is creating an interpolating polynomial basis,

$$L_i(\tau) = \prod_{\substack{j=0 \\ j \neq i}}^{N} \frac{\tau - \tau_j}{\tau_i - \tau_j}, \qquad i = 0, \dots, N. \tag{3.2}$$

where τ is the point to be interpolated and τ_j and τ_i are node positions. To get the interpolating polynomial $L(\tau)$, values at nodes x_j are multiplied by the corresponding Lagrange basis and summed

$$L(\tau) = \sum_{i=0}^{N} f_i L_i(\tau). \tag{3.3}$$

Now it is possible to interpolate a state between the discretized nodes. Note that the node locations for Legendre polynomials are arbitrary. In the actual implementation, the calculation of the polynomial basis from the definition 3.2 is numerically impractical. Therefore, the barycentric form from [13] has been used to implement these methods. The next step is to compute the definite integral using the function values at the nodes.

Gaussian Quadrature rule

A quadrature rule is the approximation of a definite integral of a function by a sum of function values at given nodes. In optimal control context Gaussian

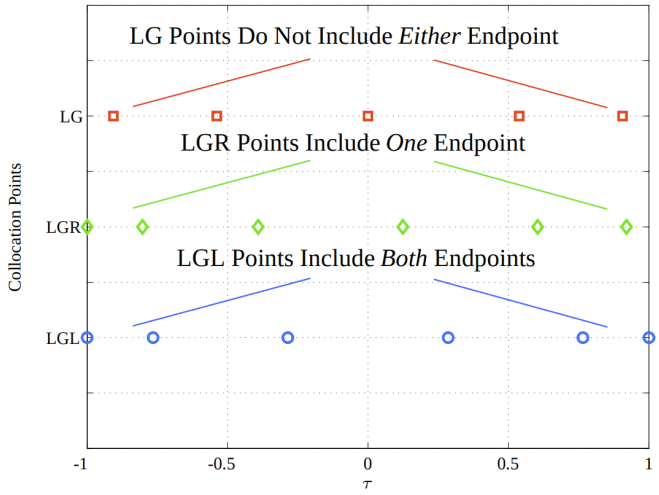


Figure 3.5: Schematic showing the differences between LGL, LGR, and LG collocation points, taken from [29]

quadrature is commonly used

$$\int_{-1}^{1} f(\tau)\,\mathrm{d}\tau \approx \sum_{i=0}^{n} w_i\, f_i, \tag{3.4}$$

where w are the Gaussian quadrature weights. A useful property of Gaussian quadrature of order N is that it approximates the integral to machine precision whenever the integrated function is a polynomial of degree at most $2N-1$. Node selection for Gauss quadrature is *not arbitrary*. In collocation methods the node positions are based on roots of Legendre polynomials. Node positions specify the type of collocation method and are created in the following manner. Let $P_N(\tau)$ denote the Legendre polynomial of degree N. Three node families are commonly used [29]:

- Legendre-Gauss: roots of $P_N(\tau)$, includes no endpoints.
- Legendre-Gauss-Radau: roots of $P_{N-1}(\tau)+P_N(\tau)$, includes one endpoint.
- Legendre-Gauss-Lobatto: roots of $\dot{P}_{N-1}(\tau)$, includes both endpoints.

3.3 Collocation at Legendre-Gauss points

Legendre-Gauss collocation methods use nodes strictly inside the interval for approximation using polynomial. With N nodes, it is possible to approximate functions up to degree $N-1$ with machine precision. States are approximated by a series

$$x_j^N(\tau) = \sum_{i=0}^{N} x_{ij}\, L_i(\tau), \tag{3.5}$$

where j is the number of the state. Then the series is differentiated using barycentric form from [13, p.513] and evaluated at k-th collocation point τ_k

$$\dot{x}_j^N(\tau_k) = \sum_{i=0}^{N} x_{ij}\,\dot{L}_i(\tau_k) = \sum_{i=0}^{N} D_{ki}\,x_{ij}, \qquad D_{ki} = \dot{L}_i(\tau_k). \tag{3.6}$$

Matrix **D** is called the *Gauss Pseudospectral differentiation matrix*. In matrix notation, it can be written

$$\mathbf{DX} = \mathbf{F}(\mathbf{X}_{\mathrm{LG}}, \mathbf{U}_{\mathrm{LG}}) \tag{3.7}$$

where $\mathbf{X_{LG}}$ is the matrix of states at Legendre-Gauss node points, $\mathbf{U_{LG}}$ is the matrix of controls at Legendre-Gauss node points, and $\mathbf{X}$ is the matrix of states at Legendre-Gauss node points and the initial point x_0

This is then used as constraints to formulate a transcription method

$$\begin{aligned} \min \quad & \Phi(\mathbf{X}_{N+1}), \\ \text{s.t.} \quad & \mathbf{DX} = \mathbf{F}(\mathbf{X}_{\mathrm{LG}}, \mathbf{U}_{\mathrm{LG}}), \\ & \mathbf{X}_{N+1} = \mathbf{X}_0 + \mathbf{w}^{\mathbf{T}}\mathbf{F}(\mathbf{X}_{\mathrm{LG}}, \mathbf{U}_{\mathrm{LG}}), \\ & \mathbf{X}_0 = \mathbf{x}_0. \end{aligned} \tag{3.8}$$

Where w are the Gauss quadrature weights. This is the differential form of the Legendre-Gauss collocation method. The integral form is described in [30].

3.4 Collocation at Legendre-Gauss-Radau points

This method is constructed in a similar manner to the previous one, with the difference that one endpoint is a collocation node. This is an advantage, as the resulting nonlinear programming formulation does not require the Gauss quadrature rule at the endpoint, and it is smaller. This method approximates functions up to degree $2N-2$ with machine precision. This is lower precision than Legendre-Gauss.

$$\begin{aligned} \min \quad & \Phi(\mathbf{X}_N) \\ \text{subject to} \quad & \mathbf{DX} = \mathbf{F}(\mathbf{X}^{\mathrm{LGR}}, \mathbf{U}^{\mathrm{LGR}}), \\ & \mathbf{X}_0 = \mathbf{x}_0. \end{aligned} \tag{3.9}$$

This is also a differential formulation.

3.5 Collocation at Legendre-Gauss-Lobatto points

The Gauss-Lobatto family includes both endpoints as collocation nodes and has a quadrature degree of exactness $2N-3$. It contains classical low-order schemes like the trapezoidal rule and Hermite–Simpson. I implement it using the Butcher tableau of LobattoIIIA implicit Runge–Kutta methods from [24].

The dynamics is collocated at N LobattoIIIA nodes $c_i \in [-1, 1]$ with coefficient matrix a_{ij} taken from the Butcher tableau. $\mathbf{X}_i$ and $\mathbf{U}_i$ are the state and control at node i, placed at $\tau_i = c_i h/2$, where h is the length of the interval. The node constraints are

$$\mathbf{X}_i = \mathbf{X}_0 + \frac{h}{2} \sum_{j=1}^{N} a_{ij}\, \mathbf{F}(\mathbf{X}_j, \mathbf{U}_j), \qquad i = 2, \dots, N, \tag{3.10}$$

where $\mathbf{X}_0$ is the left endpoint. The resulting NLP is

$$\begin{aligned} \min \quad & \Phi(\mathbf{X}_N) \\ \text{subject to} \quad & \mathbf{X}_i = \mathbf{X}_0 + \sum_{j=1}^{N} a_{ij}\, \mathbf{F}(\mathbf{X}_j, \mathbf{U}_j), \quad i = 2, \dots, N, \\ & \mathbf{X}_0 = \mathbf{x}_0. \end{aligned} \tag{3.11}$$

This is an integral formulation of a collocation method. In the book [15], the trapezoidal and Hermite-Simpson methods are written in this form.

3.5.1 h and p methods for collocation

A function can be approximated in three ways. The first uses a single high-order polynomial over the entire trajectory, these are p methods. The second uses many piecewise continuous polynomials of low and fixed order, as in [15], these are h methods. The third combines both ideas, using many polynomials whose order is allowed to vary between segments, these are hp methods. The Legendre-Gauss, Legendre-Gauss-Radau and Legendre-Gauss-Lobatto schemes presented above are the building blocks for all three categories. A single global instance gives a p method. Chaining many low-order instances on consecutive intervals with continuity at the shared endpoints gives an h method. Chaining instances of different orders gives an hp method.

Intuitively p methods should suffer from the Runge phenomenon, but in reality it is significantly suppressed. The discretization nodes are not equispaced, they are clustered toward the endpoints, which suppresses the oscillations. Methods based on roots of Chebyshev polynomials suppress the Runge phenomenon better than Legendre methods [70].

3.6 Mesh refinement methods

Mesh refinement is required to assess the accuracy of transcription, as an optimization result without a guarantee on accuracy is useless. Mesh refinement works by changing the polynomial order and the number and placement of segments. In mesh refinement, it is also important to remove segments that are unnecessary. Mesh refinement methods are divided into three categories.

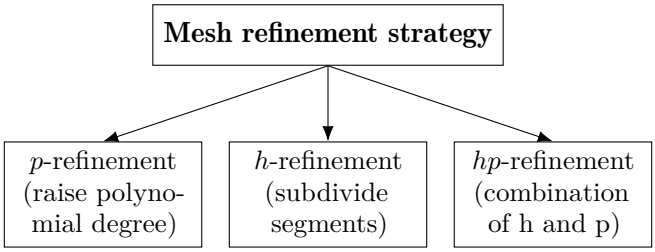


Figure 3.6: Mesh refinement categories [23, 58].

By creating mesh with proper length, density and polynomial order of segments, the NLP becomes easier to solve. Sampling too dense leads to the ratio between smallest singular value and largest singular value of the constraint jacobian to decrease [61, p. 19]. This causes numerical issues with larger number of segments. Figure 3.7 depicts this ratio. Article [65, p. 7] also examines this issue of ill conditioning. A mesh too dense leads to ill conditioning and mesh too sparse leads to incorrect result. Mesh refinement has to balance these factors.

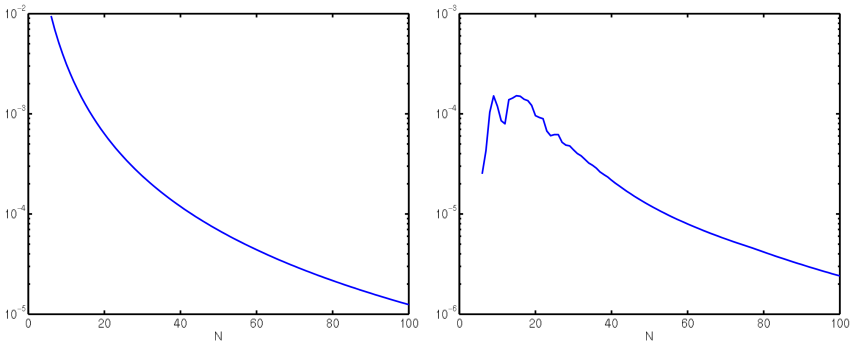


Figure 3.7: Ratio Between Minimum Singular Value and Maximum Singular Value of the Constraint Jacobian at the Solution for Various N . - Left: Linear-Quadratic OCP in Mayer Form Right: Orbit Transfer OCP. Adapted from[61, p. 19]

3.6.1 p-adaptive methods and h-adaptive methods

The p-adaptive methods work by increasing the order of the polynomial until the error is below a user-defined threshold.

The h methods are used when the solution to the optimal control problem is approximated by many intervals with a low-degree polynomial. These methods can decide to split segments into two smaller segments. They use different techniques to do this. Method described in [15] calculates error using Romberg quadrature with machine precision. Another method described in [40] uses techniques from essentially non-oscillatory (ENO) interpolation

[34] to decide where to split and remove nodes.

3.6.2 hp adaptive methods

The hp adaptive methods are state of the art in mesh refinement. They are a combination of increasing and decreasing mesh segments and increasing and decreasing the order of the approximating polynomial inside segments. One of the first hp methods and commonly cited is described in article [23] from University of Florida. They have produced numerous papers on hp-adaptive mesh refinement [9] [50]. Recent work on mesh refinement for functions with kinks [77]. They are the most complex for implementation and unfortunately I did not have the time to implement them.

3.7 Sensitivity analysis

Sensitivity analysis can be extremely useful tool when designing a race car. Knowing the sensitivity of lap-time on each of the vehicle parameters is certainly advantageous. There are two ways to perform sensitivity analysis

- Finite difference
- Differentiation methods

Finite difference is the classical approach of perturbing the parameter by couple percent and running the simulation multiple times. This is slow.

Differentiating the solution of NLP is more promising than finite difference. When the NLP problem is created with parameters θ

$$\begin{aligned} x^*(\theta) = \quad & \underset{x}{\text{argmin}} && f(x;\theta) \\ & \text{subject to} && g_i(x;\theta) \leq 0, \quad i = 1, \dots, m \\ & && A(\theta)x = b(\theta) \end{aligned} \tag{3.12}$$

differentiation methods allow differentiating the optimal solution x^* by parameters θ

$$\frac{\partial f(x^*(\theta))}{\partial \theta} = \frac{\partial f(x^*(\theta))}{\partial x^*(\theta)} \frac{\partial x^*(\theta)}{\partial \theta} \tag{3.13}$$

This is much faster than finite differences, a single backward pass yields the sensitivities of all parameters simultaneously. The explanation and implementation follow the DiffOpt.jl package [14]. However, with this approach there is no guarantee on the area where this derivative is good representation of sensitivity. With finite differences the step size can be increased to capture behavior over a larger area.

Chapter 4

Solvers for nonlinear programming

Once the objective function and constraints are created, the problem is passed to the NLP solver. Commonly used methods are Lagrange-Newton methods, these are sequential quadratic solvers and interior point solvers. These solvers are perhaps the most complex component used in this thesis, this chapter will cover them at high level, as full theory is beyond the scope. A comprehensive overview of Lagrange-Newton methods is in article [72], where the methods are broken down into key algorithmic components and unifying framework of solvers is presented. Figure 4.1 demonstrates the complexity of these methods.

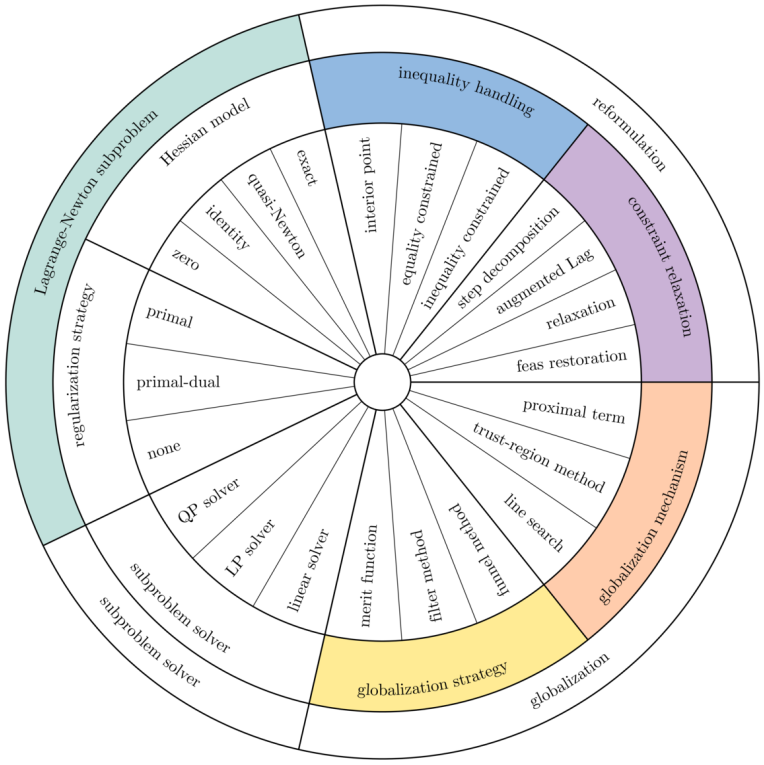


Figure 4.1: Unifying framework: the "wheel of strategies"[72]

High level overview required for this thesis of Lagrange-Newton methods is in Figure 4.2.

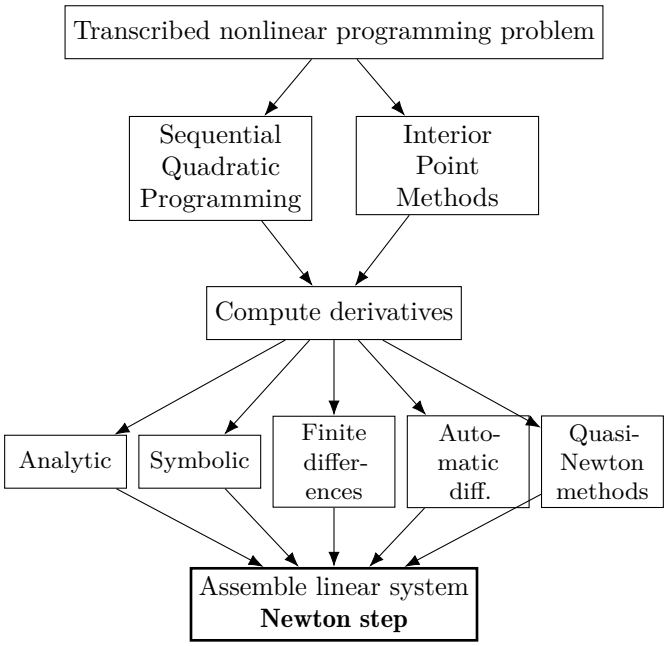


Figure 4.2: NLP solution flow

4.1 Nonlinear programming

General NLP problems can be written as:

$$\begin{aligned} \min_{\mathbf{x}\in\mathbb{R}^n} \quad & f(\mathbf{x}) \\ \text{s.t.} \quad & \mathbf{h}(\mathbf{x}) = 0 \\ & \mathbf{g}(\mathbf{x}) \leq 0 \end{aligned} \tag{4.1}$$

Here $\mathbf{x} \in \mathbb{R}^n$ is the decision-variable vector, $f : \mathbb{R}^n \to \mathbb{R}$ is the scalar objective, $\mathbf{h} : \mathbb{R}^n \to \mathbb{R}^m$ stacks the m equality constraints, and $\mathbf{g} : \mathbb{R}^n \to \mathbb{R}^p$ stacks the p inequality constraints. In the lap-time setting $\mathbf{x}$ holds the discretised states and controls, f is total lap time, $\mathbf{h}$ enforces vehicle dynamics and boundary conditions, and $\mathbf{g}$ enforces track limits, tire friction circle and actuator bounds.

4.1.1 KKT conditions

The Karush-Kuhn-Tucker (KKT) conditions are the first-order necessary conditions of optimality for the NLP (4.1). A point $\mathbf{x}^*$ together with vectors

of multipliers λ, μ is a candidate local optimum if it satisfies

$$\begin{aligned} \nabla f(\mathbf{x}) + \sum_{i=1}^{m} \lambda_i \nabla h_i(\mathbf{x}) + \sum_{i=1}^{p} \mu_i \nabla g_i(\mathbf{x}) &= 0, \\ \mathbf{h}(\mathbf{x}) &= \mathbf{0}, \\ \mathbf{g}(\mathbf{x}) &\leq \mathbf{0}, \\ \mu_i &\geq 0, \quad i = 1, \dots, p, \\ \mu_i g_i(\mathbf{x}) &= 0, \quad i = 1, \dots, p. \end{aligned} \tag{4.2}$$

The interior point Newton step in Section 4.2 solves a linearisation of these conditions, and the sensitivity analysis in Section 3.7 differentiates them with respect to model parameters.

4.2 Interior point methods: IPOPT, MadNLP

Interior point solvers or barrier solvers work by reformulating the NLP into a barrier problem at each step and solving a sequence of these barrier problems, with decreasing barrier parameter. Solver used in this thesis is IPOPT [78] and MadNLP[66].

4.2.1 Barrier problem formulation

There are multiple ways to transform NLP into barrier problem, this section will briefly describe how IPOPT does it. IPOPT article [78] describes how to transform generic NLP into form 4.3

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & \phi_\mu(\mathbf{x}) := f(\mathbf{x}) - \mu \sum_{i=1}^{n} \ln(x_i^{(i)}), \\ \text{s.t.} \quad & c(\mathbf{x}) = 0, \end{aligned} \tag{4.3}$$

inequality multipliers μ become bound multipliers z. The Lagrangian of the barrier problem is defined as

$$\mathcal{L}(\mathbf{x}, \lambda, z) := f(\mathbf{x}) + c(\mathbf{x})^\top \lambda - z. \tag{4.4}$$

Then barrier problem is further transformed into set of linear equations, a KKT linear system, that computes the Newton step:

$$\begin{bmatrix} W_k & A_k & -I \\ A_k^T & 0 & 0 \\ Z_k & 0 & X_k \end{bmatrix} \begin{bmatrix} d_x^k \\ d_\lambda^k \\ d_z^k \end{bmatrix} = - \begin{bmatrix} \nabla f(x_k) + A_k \lambda_k - z_k \\ c(x_k) \\ X_k Z_k e - \mu_j e \end{bmatrix}. \tag{4.5}$$

Here $W_k = \nabla^2 \mathcal{L}(x_k, \lambda_k, z_k)$ is the Hessian of the Lagrangian, A_k is the Jacobian of the equality constraints $c(x)$ and $\nabla f(x_k)$ is the gradient of the objective. The unknowns d_x^k, d_λ^k and d_z^k are the Newton steps in the primal variables, equality multipliers and bound multipliers; μ_j is the barrier parameter, which is driven to zero across outer iterations.

There are numerous commercial and free algorithms to solve this system of linear equations. For smaller systems it is LU decomposition, but for larger sparse problems a dedicated solver is a must. IPOPT is most commonly used with MUMPS [10] solver which is free. There are several linear commercial solvers, Harwell Subroutine Library (HSL) offers ma27 ma87 ma97 [38]. Panua Technologies offers PARDISO solver [64]. Harwell Subroutine Library solvers and PARDISO solver limited to 2 cores are available free for academic usage. These solvers run on CPU, Nvidia has linear solver CuDSS[56] which runs on GPU. Linear solvers are capable of multithreading on CPU, which needs to be explicitly turned on.

4.3 Sequential quadratic programming

Sequential quadratic programming is less commonly used for offline trajectory optimisation, usage of IPOPT prevails. celu sekciu prec

4.4 Calculation of derivatives

A significant computational burden of Lagrange-Newton methods is that they require gradient of the objective function, Jacobian of the constraints and Hessian of the Lagrangian to compute every Newton step. Computing these derivatives can be time intensive. The most universal method is finite differences, which means evaluating the function around the current point and estimating the derivative, this works even for a black box model, but the derivatives are not exact. When the model is available, symbolic differentiation can be used, however, the full symbolic expression is often too large to be evaluated effectively due to repeated chain and product rules. Automatic differentiation aims to solve the problems of previous methods, by applying the chain rule to the elementary operations of a function. A comprehensive review of automatic differentiation is given in [22].

4.5 Scaling and matrix conditioning

Common pitfall of nonlinear programming is poor scaling. This happens when the solver encounters variables, that are orders of magnitude apart, for example motor force in thousands of newtons and heading angle in radians. Newton steps in IPOPT are theoretically scale invariant, but the rest of the algorithm is not.. Even when the problem is well scaled, multipliers λ and z in (4.5) can grow very large if active-constraint gradients are nearly linearly dependent, which causes numerical difficulties in the Newton step [78, p. 21].

Similar problems happen when nonsmooth function is used in the model, like abs(), max(), branching. These have to be replaced by smooth approximation or completely reformulated.

A related issue is constraints qualification. Linear independence constraint qualification requires gradients of the active constraints to be linearly independent. If it is violated, the Jacobian of the constraints is rank deficient and linear solve may fail. This happens when for example one constraint is created twice.

4.6 Sparsity

Sparsity is a desirable property. Sparse Hessian and Jacobian matrices contain a large fraction of zero entries, which interior-point solvers exploit through specialized linear algebra. Sparsity in the constraints themselves is also important: individual constraints should involve only a small subset of the decision variables, rather than being formulated as deeply nested nonlinear expressions. Example of this is that state derivatives of vehicle at each point, can be fully determined from only 7 variables: accelerator pedal, brake pedal, steering angle, forward speed, lateral speed, yaw rate and heading. Using only these variables results in a deeply nested nonlinear expression in the constraints. Adding redundant decision variables like forces on all wheels can reduce the iterations needed and calculation time of derivatives. Counterintuitively this larger problem was faster to solve, this is elaborated on in [42, 43] Certain solvers can heavily exploit the sparsity structure, such as FATROP[73]. In source [41] Joris Gillis a CasADi developer compares IPOPT to FATROP, with **FATROP being 33 times faster on a trajectory optimisation problem.** Also, [15] describes how sparsity structure helps with faster derivatives approximation with finite difference and Quasi-Newton methods.

4.7 Initialization

NLP solvers require initial guess for all decision variables, to start solving. For this problem initial guess is obtained driving the vehicle on given track. PD controller is used to keep the car on the centerline and P controller is used to keep velocity at 5m/s. In general, it is possible to initialize all variables at zero, but with large scale nonlinear programming this approach is not used. For this problem initialization with zeros leads to singularity in equation 2.38 and is not possible at all.

4.8 Global and local optimums

This optimization problem is generally non-convex, therefore any local optimum does not guarantee global optimum. Commonly used NLP solvers, like IPOPT guarantee only local optimums and not global ones. Good initialization is basis for finding a local optimum close to the global optimum [48, §5.1, p. 11].

Part II

Implementation

Chapter 5

Frameworks for Optimal Control Problem Formulation

Process of solving optimal control problem has 3 parts. Problem definition, solving nonlinear program and optional mesh refinement. This chapter describes frameworks in which the NLP problem can be defined and solved. Framework is: programming language + algebraic modeling tool + automatic differentiation tool + mesh refining algorithm. Solving optimal control problem can be split into 3 steps described in figure 5.1

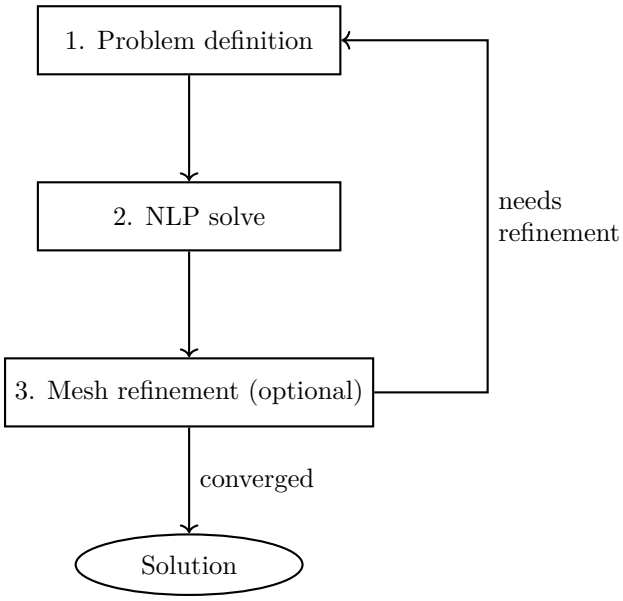


Figure 5.1: Optimal control problem solving pipeline. Adapted from [15, p. 91]

The consensus in optimal control community about tools for problem formulation is that:

> There is no shortage of optimal control software based on pseudospectral approaches. There are a number of other optimal control libraries that tackle similar kinds of problems, such as OTIS4, GPOPS-II, and CasADi. [27, 26]

5.1 CasADi

CasADi is an open-source tool for nonlinear optimization and algorithmic differentiation.[17][11]. It interfaces with C++, Python, MATLAB. It provides a framework to build the problem, and calculate the Hessian and Jacobian for NLP solver it does not have any mesh refinement algorithm. Commonly paired with Ipopt, but is compatible with large variety of solvers.

5.2 OpenMDAO + Dymos

OpenMDAO [32] was developed at NASA Glenn research center in python for evaluating aircraft concepts. Dymos implements hp mesh refinement and Lobatto and Radau collocation as default [27]. According to forum post from Julia Discourse, which may be biased towards Julia it is slower compared to InfiniteOpt in Julia [6]. It is the only open source tool I found that offers mesh refinement.

5.3 Acados

Acados is a modular open-source framework for fast embedded optimal control, developed at University of Freiburg [74]. It has C, Python, MATLAB + Simulink and Octave interface. This framework supports generating C code. It is demonstrated by numerous real-world examples, such as drone racing with *replanning*, robot navigation in cluttered environment and many others [8].

5.4 GPOPS-II

GPOPS-II [59] is perhaps the most advanced tool for optimal control, it was developed at University of Florida by Michael Patterson and Anil V. Rao. other tools allow defining the problem, but GPOPS-II is a level higher. It also includes advanced mesh refinement strategies, multi-phase OCP and automatic scaling. For solving the problem it uses Ipopt or SNOPT. It interfaces with MATLAB and C++, uses ADigator [75] for automatic differentiation. This tool is commercial and closed source. Implementation in this thesis is heavily based on work of community from University of Florida.

5.5 Julia + JuMP + InfiniteOpt / ExaModels

ExaModels, JuMP

JuMP is the go-to tool for algebraic modeling in Julia, it is mature, versatile. Supports Linear programming, quadratic programming, mixed linear programming, Nonlinear programming. ExaModels [68] is a specialized algebraic

modeling tool, it utilizes Single instruction, multiple data methods, enabling derivative evaluation on GPU accelerators. Coupled with MadNLP solver, the entire computation of nonlinear optimization can be run on a GPU. No Julia package has mesh refinement. Julia offers several framework options

1. Julia + JuMP
2. Julia + JuMP + InfiniteOpt
3. Julia + ExaModels

Although package InfiniteOpt.jl already contains implementation of various collocation methods, I chose not to use it. This decision was motivated by two considerations. Implementing the transcription methods myself provides deeper understanding of the topic. InfiniteOpt abstracts away some of the low-level control that JuMP offers, which may prevent mesh refinement implementation, usage of parametric vehicle model or sensitivity analysis.

I chose Julia + JuMP,

Chapter 6

My implementation, Julia + JuMP

I chose Julia with JuMP as the modeling environment. The toolchain had to be free and open source, fast and ergonomic. Julia targets the gap between C++ performance and Python ergonomics, and its community packages provide ready-to-use numerical tools. I also had prior experience in Python and wanted to learn Julia. My implementation is available on GitHub `https://github.com/ceserik/SLapSim.jl`.

JuMP [51] lets the user declare decision variables and constraints symbolically and uses automatic differentiation to supply first- and second-order derivatives to the solver. Through JuMP I use two NLP solvers: IPOPT [78] for the CPU path and MadNLP [67, 66] for both CPU and GPU paths. The full optimisation stack is shown in Figure 6.1.

The package InfiniteOpt.jl already implements several collocation schemes, but I chose to implement transcription myself. First, doing so gave a deeper understanding of the underlying methods. Second, InfiniteOpt abstracts away the low-level JuMP access, which may make custom mesh refinement, parametric vehicle models, and sensitivity analysis more difficult.

6.1 Sensitivity analysis

I have used DiffOpt.jl [14] for the sensitivity analysis. By adding vehicle parameters such as, mass, lift coefficient, CoG position etc. into optimization as decision variables fixed to a value by constraints. This added them to the algebraic model. Then the solution of minimum lap time can be differentiated by these parameters. Results of sensitivity analysis are in Figure 6.2.

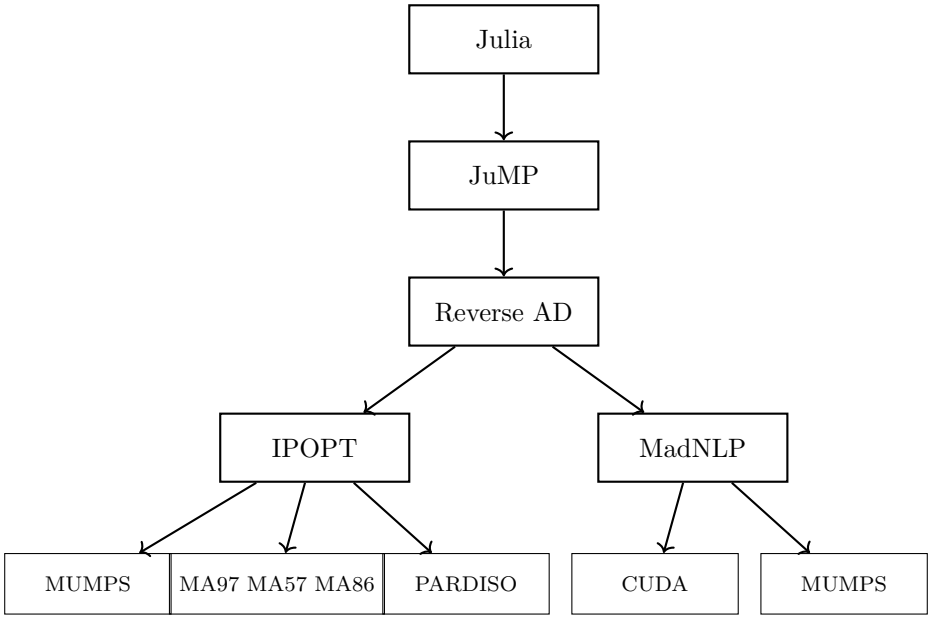


Figure 6.1: Optimization software stack

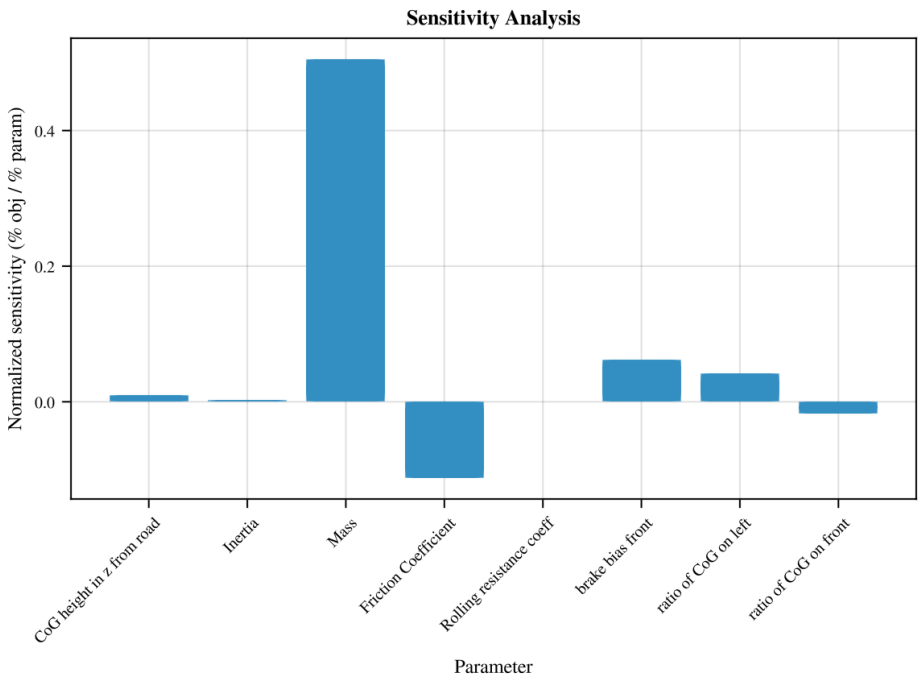


Figure 6.2: Node-interval ODE error for power-test case.

6.2 Implemented nonlinear programming solvers

Julia with JuMP offers high-level interface which allows switching NLP solvers on a single line. I have used Ipopt, MadNLP and Uno solver with preset filtersqp. Unfortunately the Uno solver was getting stuck, therefore it is not considered in this thesis.

Ipopt has built-in Hessian approximation strategy, in benchmark it proved to be inferior to automatic differentiation. Figure 6.3 shows which solving methods were implemented and tested.

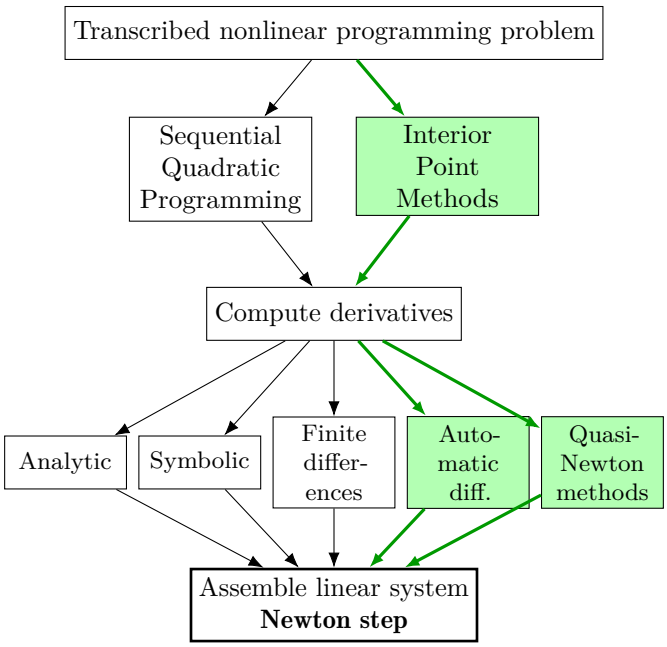


Figure 6.3: NLP solution flow with the methods implemented

6.3 Implementation of mesh refinement

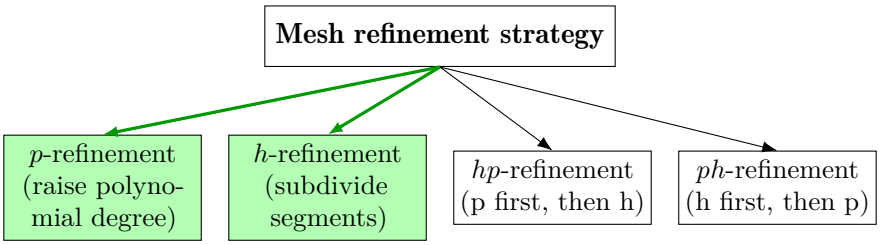


Figure 6.4: Mesh refinement layer with the strategies implemented

I have implemented p and h methods for mesh refinement. Error calculation is implemented by comparing the result of interpolated value at the end of segment and the result of simulation using Rodas4() solver. To calculate the

error at the end of each segment, the interpolated controls and initial states are plugged into Rodas4 IVP solver. Then this result is compared to the states from transcription method. If error between states in absolute value is larger than user-defined threshold, the segment is halved. This is one of the simpler estimators of discretization error.

Figure 6.5 shows that error along track is not distributed equally. Error is localized around areas with large jump in controls. These areas need finer sampling. P methods cannot focus these specific points directly, they focus entire trajectory.

The errors are concentrated around the discontinuity of controls, where the behaviour of the vehicle is more complex, requiring high-order or finer approximation see Figure 6.5. Time-optimal control of car around track has many discontinuities and therefore p methods are not a good fit for this use.

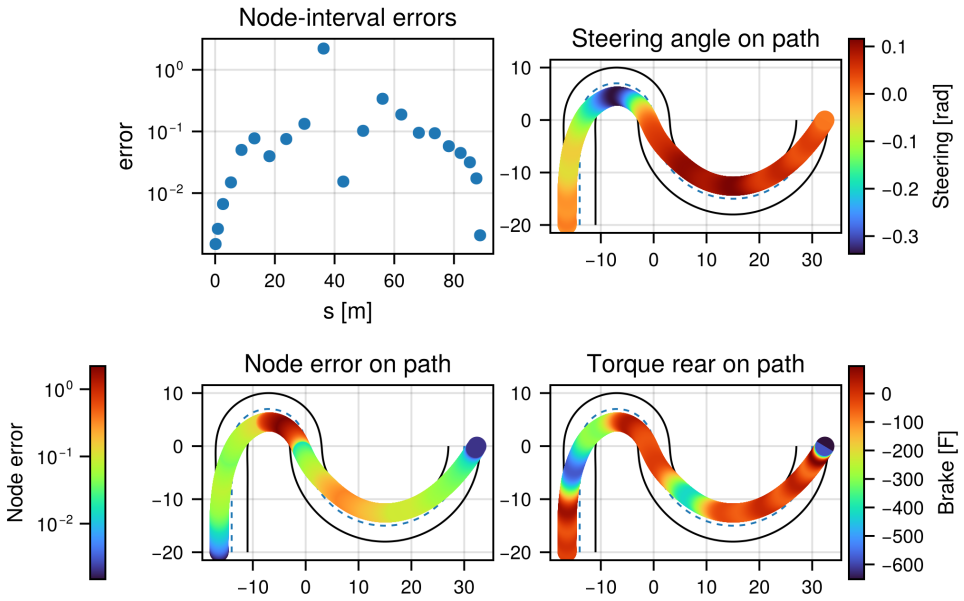


Figure 6.5: Node-interval ODE error for power-test case.

The h methods handle this better by increasing sampling density around segments with large errors. Error before refinement is on figure 6.6.

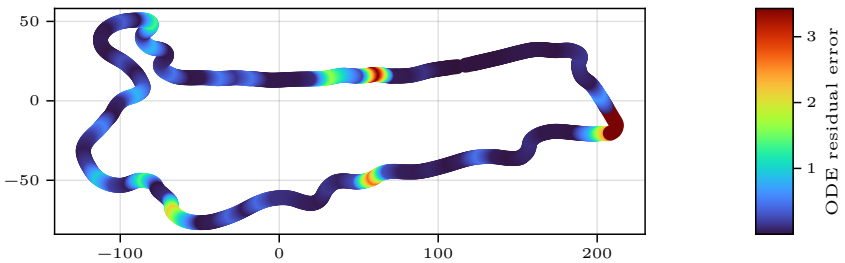


Figure 6.6: Discretization error before mesh refinement

Mesh density after refinement is on figure 6.7

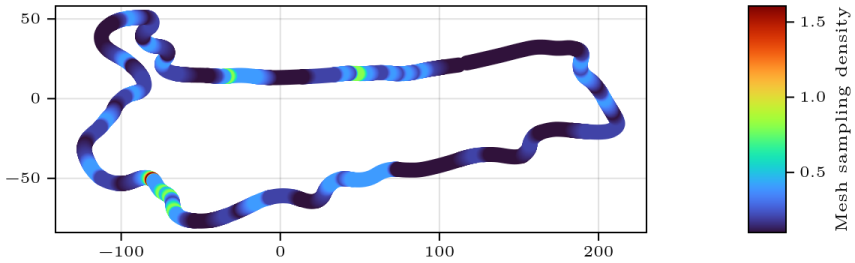


Figure 6.7: Sampling density after mesh refinement

6.4 Implementation of collocation methods

I have implemented Legendre-Gauss(LG), Legendre-Gauss-Radau(LGR) and Legendre-Gauss-Lobatto(LGL) methods. LG and LGR are implemented in differential formulation and LGL in integral formulation. Implementation supports an arbitrary number of collocation nodes. The integral formulation is chosen for LGL because, with two and three nodes, it is reduced to the trapezoidal and Hermite-Simpson methods. A significant speed-up of evaluation of automatic differentiation was obtained by creating more decision variables, thus the nonlinear constraints were less dense [42].

These methods are quite similar in their implementation in Julia, I have attempted to create a unified implementation using a single function, but I was unsuccessful. In the future, having such implementation would be beneficial to test a wider variety of collocation methods. Figure 6.8 shows methods that were implemented.

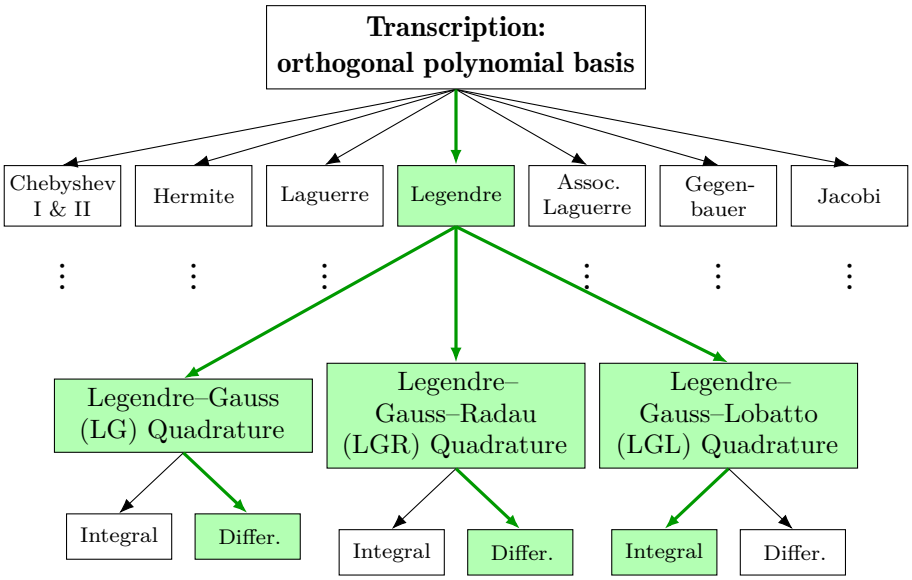


Figure 6.8: Transcription layer with the implemented methods

6.5 Implementation of models

6.5.1 Tire model implementation

Three tire models were implemented according to section 2.2. Extra decision variables were created that are equal to tire lateral and longitudinal forces, this creates less dense nonlinear expressions. Tires have these parametrizations:

Table 6.1: Tire model parameters

Linear R20

Parameter	Value
Radius R [m]	0.205
Width [m]	0.30
Max slip angle $\alpha_{\max}$ [deg]	5
Friction coefficient μ [–]	1.0
Rolling resistance c_r [–]	0.010

Pacejka R20

Parameter	Value
Radius R [m]	0.205
Width [m]	0.30
Friction coefficient μ [–]	1.0
Rolling resistance c_r [–]	0.010
Stiffness factor B	9.62
Shape factor C	2.59
Curvature factor E	1.0
Peak factor D	μF_z

Pacejka Formula E

Parameter	Value
Radius R [m]	0.33
Width [m]	0.30
Friction coefficient μ [–]	1.0
Rolling resistance c_r [–]	0.010
Stiffness factor B	9.62
Shape factor C	2.59
Curvature factor E	1.0
Peak factor D	$\mu F_z \left(1 - \frac{0.0813\, F_z}{3000}\right)$

6.5.2 Implementation of vehicles

Four vehicles were implemented. The simplest is the single-track Formula Student vehicle, with a static suspension 2.6.1 and a linear tire 2.2.1. The controls are the braking force, the rear motor torque, and the steering of the front wheels.

The twin-track Formula Student vehicle has a simplified Magic Formula tire model 2.2.2 without the degressive behavior and a static suspension 2.6.1. The controls are the torque on the front motors, the torque on the rear motors, the braking force, and the steering of the front wheels. It is implemented according to [1].

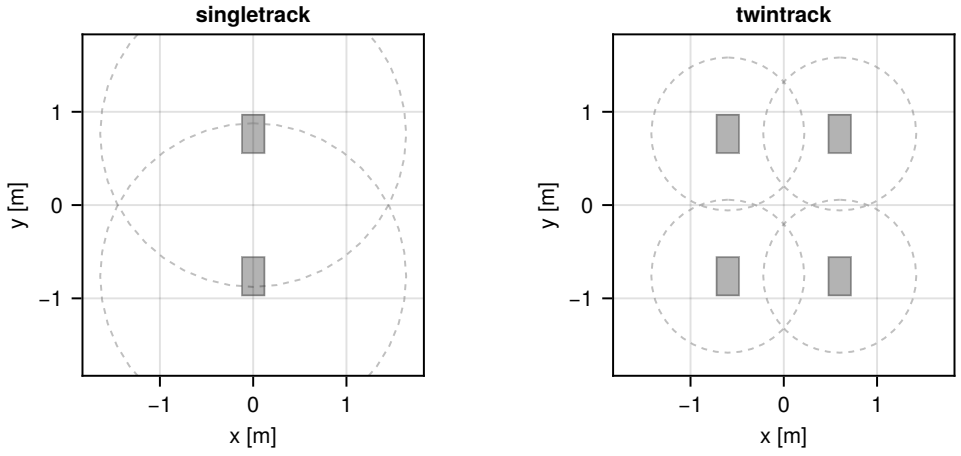
The twin-track Formula E model has a Magic Formula tire model 2.2.2 with degressive behavior and a dynamic suspension. The controls are the torque on the rear wheels, the braking force, and the steering of the front wheels.

The bus model has a scaled-up Magic Formula tire model and a dummy suspension that splits the normal load equally. The controls are the torque

on the middle axle, the independent steering of the front and rear wheels, and the braking force.

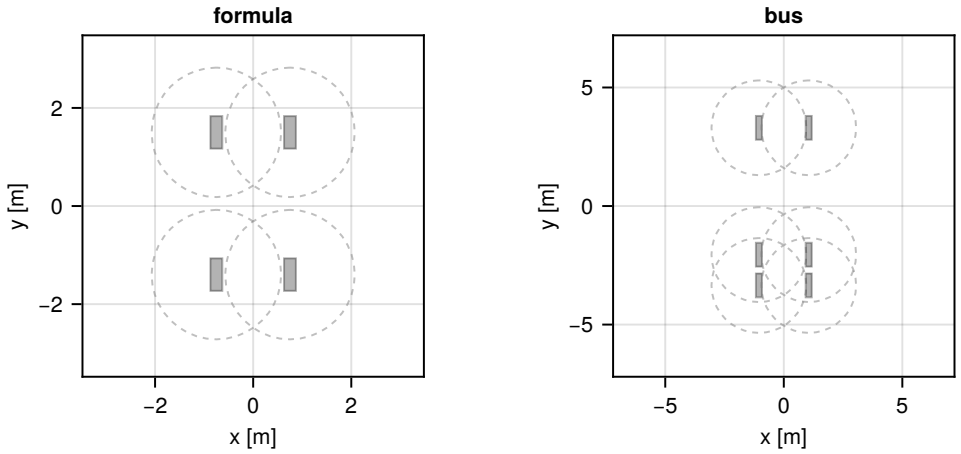
Table 6.2: Vehicle parameters used in this thesis

Parameter	Singletrack	FS Twintrack	Formula E	Bus
Mass m [kg]	280	280	1200	12000
Inertia I_z [kg m^2]	100	100	1260	30000
Wheelbase L [m]	1.525	1.525	2.9	6.0
Track w [m]	1.2	1.2	1.5	2.1
CoG ratio front [–]	0.50	0.50	0.482	0.45
Lift coefficient C_L [–]	5.0	5.0	−2.7	−0.2
Drag coefficient C_D [–]	2.0	2.0	1.4	6.0
Power limit $P_{\max}$ [kW]	80	80	270	300
Brake bias front [–]	0.60	0.60	0.70	0.60
Max brake force [kN]	20	20	20	500
Velocity range [m/s]	[3,60]	[3,60]	[2,60]	[2,30]
Wheels	2	4	4	6
Driven motors	2	4	1 (shared)	6
Tire model	linear R20	Pacejka R20	Pacejka FE	Pacejka (scaled)
Suspension	Static	Static	Dynamic	dummy 6-wheel



(a) : Singletrack **(b) :** Formula Student twintrack

Figure 6.9: Single-track and Formula Student twintrack vehicle models.

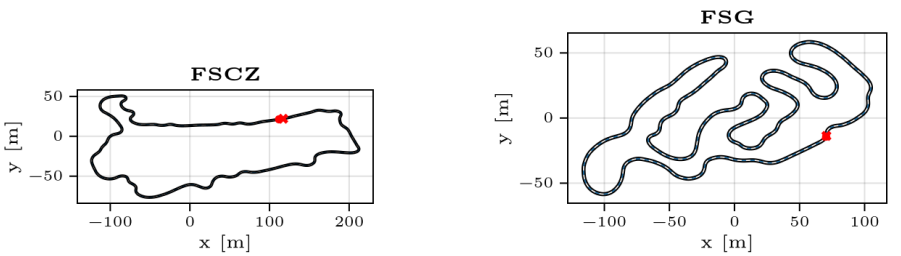


(a) : Formula E **(b) :** Bus

Figure 6.10: Formula E and Bus vehicle models.

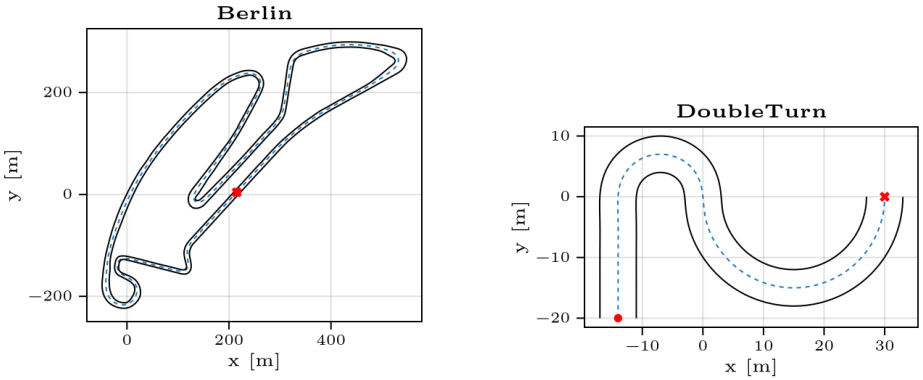
6.6 Benchmark tracks

The following tracks are used in the benchmark in Section 7. They cover real Formula Student layouts (FSCZ, FSG), a Formula E street circuit (Berlin) and two synthetic geometries (DoubleTurn, FigureEight).



(a) : Formula Student Czech track **(b) :** Formula Student Germany track

Figure 6.11: Formula Student benchmark tracks.



(a) : Berlin (Formula E) **(b) :** DoubleTurn

Figure 6.12: Berlin Formula E circuit and the DoubleTurn synthetic track.

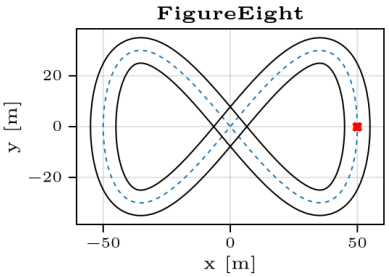


Figure 6.13: FigureEight synthetic track.

6.7 Example optimization problem

After transcription the OCP becomes a nonlinear program over the state and control samples at the collocation nodes,

$$
\begin{aligned}
\min_{\mathbf{X},\mathbf{U}} \quad & t_f \\
\text{s.t.} \quad & \mathbf{DX} = \mathbf{F}(\mathbf{X},\mathbf{U}), && \text{collocated dynamics} \\
& \mathbf{G}(\mathbf{X},\mathbf{U}) \leq 0, && \text{path constraints at nodes} \\
& \mathbf{X}_0 = \mathbf{x}_0, \quad \mathbf{X}_N = \mathbf{x}_f, && \text{boundary conditions} \\
& \mathbf{X}_{\text{lb}} \leq \mathbf{X} \leq \mathbf{X}_{\text{ub}}, && \text{state bounds} \\
& \mathbf{U}_{\text{lb}} \leq \mathbf{U} \leq \mathbf{U}_{\text{ub}}. && \text{control bounds}
\end{aligned}
\tag{6.1}
$$

the objective is to minimize the final time.

Part III

Results

Chapter 7

Comparison of methods

The performance of NLP solvers and frameworks is often tested on benchmark problems, like the CUTEst [31] dataset. I have created my own dataset for this purpose. As written on Ipopt website: Ipopt has many (probably too many) options [21] that can be adjusted, and that is true for all solvers. Some of differences in this benchmark may be due to suboptimal setup of the solver. Mesh refinement method in these tests is the h method with error threshold $10^{-}1$ Tests were performed on Ryzen 9 9950X using 32 cores by setting environment variable `OMP_NUM_THREADS` to 32 and MadNLP ran on Nvidia RTX 5070. This benchmark tests the time to solve and solve failure rate for all implemented vehicle variants on benchmarking tracks. Videos of some of the benchmarks are available on the YouTube playlist [20].

7.1 Benchmark of collocation methods

This benchmark tests collocation methods. Legendre and Lobatto methods failed in one of 14 tests Radau did not fail, but took little longer than Lobatto method. Interestingly the lap-time seems to vary in some cases by around 0.3 second on same track and same vehicle with different transcription method.

transcription	runs	ok	fail	rate	mean_s	med_s	min_s	max_s
Legendre	14	14	0	100.0000	167.1341	25.1476	0.8392	1688.6196
Lobatto IIIA	14	14	0	100.0000	58.1844	16.6429	0.3494	546.0076
Radau	14	14	0	100.0000	183.1201	14.4853	0.7666	2272.8209

Table 7.2: Benchmark summary statistics by transcription variant.

track	car	transcription	ok	status	time_s	obj
FSCZ	singletrack	Radau	yes	solved	9.6201	47.1377
FSCZ	singletrack	Legendre	yes	solved	26.7235	47.2675
FSCZ	singletrack	Lobatto IIIA	yes	solved	17.8552	47.5837
FSCZ	twintrack	Radau	yes	solved	29.3340	53.1709
FSCZ	twintrack	Legendre	yes	solved	61.1884	53.4512
FSCZ	twintrack	Lobatto IIIA	yes	solved	25.6206	53.9954
Berlin	singletrack	Radau	yes	solved	22.0242	79.4397
Berlin	singletrack	Legendre	yes	solved	23.5717	79.4492
Berlin	singletrack	Lobatto IIIA	yes	solved	52.4490	79.4002
Berlin	twintrack	Radau	yes	solved	73.7309	77.7238

Berlin	twintrack	Legendre	yes	solved	46.3706	78.0661
Berlin	twintrack	Lobatto IIIA	yes	solved	32.2219	77.7762
Berlin	formula	Radau	yes	solved	72.6660	87.9109
Berlin	formula	Legendre	yes	solved	407.0615	88.9009
Berlin	formula	Lobatto IIIA	yes	solved	63.9919	87.9114
Berlin	bus	Radau	yes	solved	2272.8209	106.6878
Berlin	bus	Legendre	yes	solved	1688.6196	106.7405
Berlin	bus	Lobatto IIIA	yes	solved	546.0076	106.1627
FSG	singletrack	Radau	yes	solved	11.4388	53.8713
FSG	singletrack	Legendre	yes	solved	28.6402	53.9862
FSG	singletrack	Lobatto IIIA	yes	solved	15.4307	53.4303
FSG	twintrack	Radau	yes	solved	46.4263	64.6331
FSG	twintrack	Legendre	yes	solved	36.2408	64.7986
FSG	twintrack	Lobatto IIIA	yes	solved	48.9636	64.2024
DoubleTurn	singletrack	Radau	yes	solved	1.2873	5.6450
DoubleTurn	singletrack	Legendre	yes	solved	0.8392	5.6384
DoubleTurn	singletrack	Lobatto IIIA	yes	solved	0.3494	5.6922
DoubleTurn	twintrack	Radau	yes	solved	0.7666	6.5474
DoubleTurn	twintrack	Legendre	yes	solved	0.8690	6.5527
DoubleTurn	twintrack	Lobatto IIIA	yes	solved	0.6170	6.7016
DoubleTurn	bus	Radau	yes	solved	1.6908	8.9032
DoubleTurn	bus	Legendre	yes	solved	1.4636	8.9150
DoubleTurn	bus	Lobatto IIIA	yes	solved	2.0459	8.8830
FigureEight	singletrack	Radau	yes	solved	1.1828	14.9425
FigureEight	singletrack	Legendre	yes	solved	2.0092	14.9482
FigureEight	singletrack	Lobatto IIIA	yes	solved	2.8800	14.8949
FigureEight	twintrack	Radau	yes	solved	3.1612	16.6011
FigureEight	twintrack	Legendre	yes	solved	7.5868	16.6084
FigureEight	twintrack	Lobatto IIIA	yes	solved	1.9091	16.6481
FigureEight	bus	Radau	yes	solved	17.5317	22.8455
FigureEight	bus	Legendre	yes	solved	8.6937	22.8542
FigureEight	bus	Lobatto IIIA	yes	solved	4.2393	22.4779

Table 7.4: Benchmark results by track, vehicle model, and transcription variant.

7.2 Benchmark of linear solvers

Surprisingly free MUMPS solver performed better than solvers from proprietary Harwell Subroutine Library. In this test I did not include PARDISO as my license was limited only to two cores.

linear solver	runs	ok	fail	rate	mean_s	med_s	min_s	max_s
ma27	14	14	0	100.0000	261.8886	14.3300	0.3385	3390.5082
ma57	14	14	0	100.0000	80.8659	14.0413	0.7563	839.6873
ma97	14	13	1	92.8571	22.9438	12.2251	0.3456	80.9731
mumps	14	14	0	100.0000	182.9820	14.3890	0.3590	2264.3630

Table 7.6: Linear solver benchmark summary statistics.

track	car	linear solver	ok	status	time_s	obj
FSCZ	singletrack	mumps	yes	solved	9.5894	47.1377
FSCZ	singletrack	ma57	yes	solved	11.3044	47.1377
FSCZ	singletrack	ma27	yes	solved	9.3451	47.1377

FSCZ	singletrack	ma97	yes	solved	10.5403	47.1377
FSCZ	twintrack	mumps	yes	solved	29.3824	53.1709
FSCZ	twintrack	ma57	yes	solved	26.9544	53.1709
FSCZ	twintrack	ma27	yes	solved	28.0511	53.1709
FSCZ	twintrack	ma97	yes	solved	26.3581	53.1709
Berlin	singletrack	mumps	yes	solved	25.9843	79.4397
Berlin	singletrack	ma57	yes	solved	18.4504	79.4397
Berlin	singletrack	ma27	yes	solved	19.2238	79.4397
Berlin	singletrack	ma97	yes	solved	27.0323	79.4397
Berlin	twintrack	mumps	yes	solved	75.4742	77.7238
Berlin	twintrack	ma57	yes	solved	58.6173	77.7238
Berlin	twintrack	ma27	yes	solved	67.6959	77.7238
Berlin	twintrack	ma97	yes	solved	80.9731	77.7238
Berlin	formula	mumps	yes	solved	73.7649	87.9109
Berlin	formula	ma57	yes	solved	99.8552	87.9109
Berlin	formula	ma27	yes	solved	72.8937	87.9109
Berlin	formula	ma97	yes	solved	78.1892	87.9103
Berlin	bus	mumps	yes	solved	2264.3630	106.6878
Berlin	bus	ma57	yes	solved	839.6873	106.6882
Berlin	bus	ma27	yes	solved	3390.5082	106.7080
Berlin	bus	ma97	no	iteration limit	605.6959	NaN
FSG	singletrack	mumps	yes	solved	11.5927	53.8713
FSG	singletrack	ma57	yes	solved	12.5714	53.8713
FSG	singletrack	ma27	yes	solved	12.9429	53.8713
FSG	singletrack	ma97	yes	solved	12.2251	53.8713
FSG	twintrack	mumps	yes	solved	47.1838	64.6331
FSG	twintrack	ma57	yes	solved	41.5523	64.6331
FSG	twintrack	ma27	yes	solved	44.1811	64.6331
FSG	twintrack	ma97	yes	solved	40.6957	64.6380
DoubleTurn	singletrack	mumps	yes	solved	0.3590	5.6450
DoubleTurn	singletrack	ma57	yes	solved	1.3340	5.6450
DoubleTurn	singletrack	ma27	yes	solved	0.3385	5.6450
DoubleTurn	singletrack	ma97	yes	solved	0.3456	5.6450
DoubleTurn	twintrack	mumps	yes	solved	0.6577	6.5474
DoubleTurn	twintrack	ma57	yes	solved	0.8025	6.5474
DoubleTurn	twintrack	ma27	yes	solved	0.6423	6.5474
DoubleTurn	twintrack	ma97	yes	solved	0.5053	6.5488
DoubleTurn	bus	mumps	yes	solved	0.9267	8.9032
DoubleTurn	bus	ma57	yes	solved	0.7563	8.9032
DoubleTurn	bus	ma27	yes	solved	0.8304	8.9032
DoubleTurn	bus	ma97	yes	solved	0.7686	8.9032
FigureEight	singletrack	mumps	yes	solved	1.2690	14.9425
FigureEight	singletrack	ma57	yes	solved	1.9768	14.9425
FigureEight	singletrack	ma27	yes	solved	1.2471	14.9425
FigureEight	singletrack	ma97	yes	solved	1.0779	14.9425
FigureEight	twintrack	mumps	yes	solved	4.0156	16.6011
FigureEight	twintrack	ma57	yes	solved	2.7487	16.6011
FigureEight	twintrack	ma27	yes	solved	2.8235	16.6011
FigureEight	twintrack	ma97	yes	solved	3.7290	16.6011
FigureEight	bus	mumps	yes	solved	17.1854	22.8455
FigureEight	bus	ma57	yes	solved	15.5112	22.8455
FigureEight	bus	ma27	yes	solved	15.7170	22.8455
FigureEight	bus	ma97	yes	solved	15.8290	22.8455

Table 7.8: Linear solver benchmark results by track, vehicle model, and transcription variant.

7.3 Benchmark of NLP solvers

I have tested Ipopt with MUMPS and MadNLP with MUMPS and with CuDSS solver. These solvers were suspiciously similar in computation time on CPU and GPU was slower. with madnlp-gpu variant the automatic differentiation was calculated on CPU using JuMP and the solver ran on GPU [69], so it was not a 100% GPU solution.

NLP solver	runs	ok	fail	rate	mean_s	med_s	min_s	max_s
ipopt-mumps	14	14	0	100.0000	194.9796	14.2172	0.3935	2443.6578
madnlp-cpu	14	13	1	92.8571	23.3577	14.0445	0.3068	94.4898
madnlp-gpu	14	13	1	92.8571	30.0594	14.8362	0.6094	137.7783

Table 7.10: NLP solver / backend benchmark summary statistics (IPOPT-MUMPS, MadNLP-CPU, MadNLP-GPU).

track	car	NLP solver	ok	status	time_s	obj
FSCZ	singletrack	ipopt-mumps	yes	solved	10.4205	47.1377
FSCZ	singletrack	madnlp-cpu	yes	solved	18.3293	47.1377
FSCZ	singletrack	madnlp-gpu	yes	solved	66.7619	47.1363
FSCZ	twintrack	ipopt-mumps	yes	solved	29.6126	53.1709
FSCZ	twintrack	madnlp-cpu	yes	solved	31.7456	53.1709
FSCZ	twintrack	madnlp-gpu	yes	solved	60.5383	53.2287
Berlin	singletrack	ipopt-mumps	yes	solved	16.7130	79.4397
Berlin	singletrack	madnlp-cpu	yes	solved	14.0445	79.4397
Berlin	singletrack	madnlp-gpu	yes	solved	14.8362	79.4358
Berlin	twintrack	ipopt-mumps	yes	solved	72.1841	77.7238
Berlin	twintrack	madnlp-cpu	yes	solved	94.4898	77.7237
Berlin	twintrack	madnlp-gpu	yes	solved	23.4224	77.7201
Berlin	formula	ipopt-mumps	yes	solved	71.9810	87.9109
Berlin	formula	madnlp-cpu	yes	solved	81.3535	87.9111
Berlin	formula	madnlp-gpu	yes	almost locally solved	137.7783	87.9093
Berlin	bus	ipopt-mumps	yes	solved	2443.6578	106.6937
Berlin	bus	madnlp-cpu	no	iteration limit	1178.0533	NaN
Berlin	bus	madnlp-gpu	no	iteration limit	4431.9955	NaN
FSG	singletrack	ipopt-mumps	yes	solved	11.7215	53.8713
FSG	singletrack	madnlp-cpu	yes	solved	12.8528	53.8713
FSG	singletrack	madnlp-gpu	yes	solved	12.2274	53.8696
FSG	twintrack	ipopt-mumps	yes	solved	47.4247	64.6331
FSG	twintrack	madnlp-cpu	yes	solved	29.7381	64.5931
FSG	twintrack	madnlp-gpu	yes	solved	29.5928	64.5912
DoubleTurn	singletrack	ipopt-mumps	yes	solved	0.3935	5.6450
DoubleTurn	singletrack	madnlp-cpu	yes	solved	0.3068	5.6450
DoubleTurn	singletrack	madnlp-gpu	yes	solved	0.6094	5.6449
DoubleTurn	twintrack	ipopt-mumps	yes	solved	0.6312	6.5474
DoubleTurn	twintrack	madnlp-cpu	yes	solved	0.7809	6.5488
DoubleTurn	twintrack	madnlp-gpu	yes	solved	0.7918	6.5487
DoubleTurn	bus	ipopt-mumps	yes	solved	1.8919	8.9032
DoubleTurn	bus	madnlp-cpu	yes	solved	0.7989	8.9032
DoubleTurn	bus	madnlp-gpu	yes	solved	1.4556	8.9031
FigureEight	singletrack	ipopt-mumps	yes	solved	1.4011	14.9425
FigureEight	singletrack	madnlp-cpu	yes	solved	1.2097	14.9425
FigureEight	singletrack	madnlp-gpu	yes	solved	1.2459	14.9419
FigureEight	twintrack	ipopt-mumps	yes	solved	4.1659	16.6011
FigureEight	twintrack	madnlp-cpu	yes	solved	2.4902	16.6011

FigureEight	twintrack	madnlp-gpu	yes	solved	2.6734	16.5974
FigureEight	bus	ipopt-mumps	yes	solved	17.5150	22.8455
FigureEight	bus	madnlp-cpu	yes	solved	15.5101	22.9008
FigureEight	bus	madnlp-gpu	yes	solved	38.8391	22.9871

Table 7.12: NLP solver / backend benchmark results by track, vehicle model, and transcription variant.

7.4 Benchmark of derivatives computation

This section compares two ways to provide the Hessian to Ipopt. Exact second-order derivatives obtained through JuMP's automatic differentiation are compared against Ipopt's built-in limited-memory quasi-Newton approximation. Exact Hessian was faster in all tested cases. The quasi-Newton approximation also failed to converge in time on some of the harder problems.

hessian mode	runs	ok	fail	rate	mean_s	med_s	min_s	max_s
ipopt-exact	14	14	0	100.0000	196.4979	14.3078	0.3420	2459.4467
ipopt-lbfgs	14	7	7	50.0000	9.9792	7.3379	0.4952	25.2870

Table 7.14: Hessian approximation benchmark summary statistics (exact vs. limited-memory quasi-Newton).

track	car	hessian mode	ok	status	time_s	obj
FSCZ	singletrack	ipopt-exact	yes	solved	10.6736	47.1377
FSCZ	singletrack	ipopt-lbfgs	yes	almost locally solved	16.1547	47.1377
FSCZ	twintrack	ipopt-exact	yes	solved	30.6336	53.1709
FSCZ	twintrack	ipopt-lbfgs	no	iteration limit	315.6241	NaN
Berlin	singletrack	ipopt-exact	yes	solved	17.1804	79.4397
Berlin	singletrack	ipopt-lbfgs	yes	almost locally solved	25.2870	79.4397
Berlin	twintrack	ipopt-exact	yes	solved	71.7344	77.7238
Berlin	twintrack	ipopt-lbfgs	no	iteration limit	1269.5613	NaN
Berlin	formula	ipopt-exact	yes	solved	74.9029	87.9109
Berlin	formula	ipopt-lbfgs	no	iteration limit	338.9663	NaN
Berlin	bus	ipopt-exact	yes	solved	2459.4467	106.6937
Berlin	bus	ipopt-lbfgs	no	iteration limit	1319.7326	NaN
FSG	singletrack	ipopt-exact	yes	solved	11.4352	53.8713
FSG	singletrack	ipopt-lbfgs	yes	solved	16.2359	53.8713
FSG	twintrack	ipopt-exact	yes	solved	50.1417	64.6331
FSG	twintrack	ipopt-lbfgs	no	iteration limit	334.7838	NaN
DoubleTurn	singletrack	ipopt-exact	yes	solved	0.3420	5.6450
DoubleTurn	singletrack	ipopt-lbfgs	yes	solved	0.4952	5.6450
DoubleTurn	twintrack	ipopt-exact	yes	solved	0.6957	6.5474
DoubleTurn	twintrack	ipopt-lbfgs	yes	solved	1.7747	6.5488
DoubleTurn	bus	ipopt-exact	yes	solved	0.8589	8.9032
DoubleTurn	bus	ipopt-lbfgs	yes	solved	7.3379	8.9041
FigureEight	singletrack	ipopt-exact	yes	solved	1.0858	14.9425
FigureEight	singletrack	ipopt-lbfgs	yes	almost locally solved	2.5691	14.9425
FigureEight	twintrack	ipopt-exact	yes	solved	4.0830	16.6011
FigureEight	twintrack	ipopt-lbfgs	no	iteration limit	52.9105	NaN
FigureEight	bus	ipopt-exact	yes	solved	17.7560	22.8455
FigureEight	bus	ipopt-lbfgs	no	iteration limit	150.1921	NaN

Table 7.16: Hessian approximation benchmark results by track, vehicle model, and transcription variant.

Chapter 8

Conclusion

Goal of investigation of available numerical methods was accomplished, the topic is very deep with all of the subproblems: dynamic system modeling, transcription, algebraic modeling, mesh refinement, NLP solving, sparse linear solving, sensitivity analysis and frameworks that encompass them being infinitely deep I CAN SPEND INFITE TIME LEARNING THEM. It was difficult to understand these topics in meaning ful way without spend too much time on a single topic.
More investigation would be nice into differentiate orthogonal polynomials, Chebyshev type I. polynomials are often used with high order due to their good suppression of Runge phenomena.
As for NLP solvers a better set of options can be found for Ipopt by investigating why the solver fails in some cases, also it may make sense to try Uno solver [72] and construct a new specialized solver for this problem, but that would take enormous amount of time.
Building a model for optimization was difficult, as debugging it is difficult, when the grid is sampled too densely or different thins are not good, Ipopt fails to converge. I wasn't using any tool to find which constraint is causing this, debugging could be improved by writing/using a tool which would tell which constraint is causing issues. This would significantly decrease the development time.
h method implemeted is not good very bad better needs to be implementd the rror estimation is only placeholder
As for implementation, Julia was interesting language to learn, it offers blazing fast execution, but that comes with caveats. Debugging is way more complicated. When working on a smaller project, the debugging is not such a pain, but when the project is bigger debugging is not user friendly. Debugging in python and MATLAB is way easier. Julia's main selling point is that it is just in time compiled language, rivalling C after compilation. That is true BUT you have to know what you are doing, if you don't, it can be as slow as python. Also JuMP was taking 90% of NLP solving time on calculating the derivatives, when constraints had dense expressions [42, **?**], so only 10% by the actual solver, CasADi seems not to have this issue also implementation using automatic differentiation from ExaModels [66] on GPU would be beneficial to experiment with.

One thing which was not a goal but, would be super useful if i managed to do a validation of sensitivity analysis on real world data. That is driving vehicle on track, measuring time, loading sand bags (and/or myself as passenger) as extra weight into the car and driving again. I wanted to do it for a long time but there was not enough of time.
At the begging of this project I wanted a more complex model, I wanted to go deeper with complexity than my bachelor thesis. That is add pitch, roll and add suspension with damper and spring for each wheel, add Equivalent circuit model 1RC for the accumulator, properly implement energy accumulation, add road bank and inclination. Investigate the possible implementations of losses from Section 2.9 and using softmax or different approximations. Also this simulation does not account for gearboxes with discrete stages, and no combustion engine.
Believing 100% this tool and designing car according to it would most likely lead to undriveable vehicle, as objective of time minimization doesn't account for stability and driveability, [60] deals also with this. The optimal driving created by NLP is unachievable by real driver, as the vehicle is on its limit and one mistake would overload tires and cause failure.
the comparison is biased towards high number, the total time should not be relative to total time of all solver or timeof longest solve. Also median is not telling much with so few samples.
Automated behncmarking is a must becasue small canges can break many things and you may notice the simulation crashing long after iplementaton
implement softmax and max approximaitons

Appendices

Appendix A

Bibliography

[1] 2026 Class Project: Race Car Minimum Lap Time - AE6310 Course Notes. https://smdogroup.github.io/ae6310/jupyter/2026_project/race_car_problem.html.

[2] A Computationally Efficient Framework for Free-trajectory Minimum-lap-time Optimization of Racing Cars. https://arxiv.org/html/2511.13522v1.

[3] OptimumLap | OptimumG.

[4] Solving ordinary differential equations I. nonstiff problems. *Mathematics and Computers in Simulation*, 29(5):447, October 1987.

[5] Solving ordinary differential equations I. nonstiff problems. *Mathematics and Computers in Simulation*, 29(5):447, October 1987.

[6] Does anybody know OpenMDAO - Specific Domains / Optimization (Mathematical). https://discourse.julialang.org/t/does-anybody-know-openmdao/122840, November 2024.

[7] Coin-or/CppAD. COIN-OR Foundation, May 2026.

[8] acados Developers. acados documentation: real-world examples. `https://docs.acados.org/real_world_examples/index.html`, 2026. Accessed: 2026-05-14.

[9] Yunus Agamawi and Anil Rao. CGPOPS: A C++ Software for Solving Multiple-Phase Optimal Control Problems Using Adaptive Gaussian Quadrature Collocation and Sparse Nonlinear Programming. *ACM Transactions on Mathematical Software*, 46, May 2020.

[10] P.R. Amestoy, I. S. Duff, J. Koster, and J.-Y. L'Excellent. A fully asynchronous multifrontal solver using distributed dynamic scheduling. *SIAM Journal on Matrix Analysis and Applications*, 23(1):15–41, 2001.

[11] Joel A E Andersson, Joris Gillis, Greg Horn, James B Rawlings, and Moritz Diehl. CasADi – A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019.

[12] Uri M. Ascher, Robert M. M. Mattheij, and Robert D. Russell. *Numerical Solution of Boundary Value Problems for Ordinary Differential Equations*. Classics in Applied Mathematics. SIAM, 1995.

[13] Jean-Paul Berrut and Lloyd N. Trefethen. Barycentric Lagrange Interpolation. *SIAM Review*, 46(3):501–517, January 2004.

[14] Mathieu Besançon, Joaquim Dias Garcia, Benoît Legat, and Akshay Sharma. Flexible differentiable optimization via model transformations. *INFORMS Journal on Computing*, 36(2):456–478, 2023.

[15] John T. Betts. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*. Society for Industrial and Applied Mathematics, second edition, January 2010.

[16] Forbes T. Brown. *Engineering System Dynamics: A Unified Graph-Centered Approach, Second Edition*. CRC Press, Boca Raton, 2 edition, August 2006.

[17] CasADi Developers. CasADi – an open-source tool for nonlinear optimization and algorithmic differentiation. `https://web.casadi.org/`, 2026. Accessed: 2026-05-14.

[18] D. Casanova. On minimum time vehicle manoeuvring: The theoretical optimal lap. November 2000.

[19] Richard Ciglanský. Analysis of vehicle components and their impact on lap-time.

[20] Richard Ciglanský. SLapSim benchmark animations. YouTube playlist, `https://youtube.com/playlist?list=PL_TqVFNheNEL-B8Fof0HaNX4ybwSa8F1H`, 2026. Accessed 2026-05-21.

[21] COIN-OR Foundation. Ipopt options reference. `https://coin-or.github.io/Ipopt/OPTIONS.html`, 2026. Accessed: 2026-05-18.

[22] Daniel Cortild, J Haastert, A Sanabria, and F Voronine. *A Brief Review of Automatic Differentiation*. March 2023.

[23] Christopher L. Darby, William W. Hager, and Anil V. Rao. An *hp* -adaptive pseudospectral method for solving optimal control problems. *Optimal Control Applications and Methods*, 32(4):476–502, July 2011.

[24] Byron Ehle. On Padé approximations to the exponential function and A-stable methods for the numerical solution of initial value problems.

[25] T.M. El-Gindy, H.M. El-Hawary, M.S. Salim, and M. El-Kady. A Chebyshev approximation for solving optimal control problems. *Computers & Mathematics with Applications*, 29(6):35–45, 1995.

[26] Rob Falck, Justin S. Gray, Kaushik Ponnapalli, and Ted Wright. Dymos: A Python package for optimal control of multidisciplinary systems. `https://openmdao.org/dymos/docs/1.9.0/`, 2024. Version 1.9.0, accessed 2026-05-20.

[27] Robert Falck, Justin S. Gray, Kaushik Ponnapalli, and Ted Wright. dymos: A python package for optimal control of multidisciplinary systems. *Journal of Open Source Software*, 6(59):2809, 2021.

[28] Liang Fang, Stefan Vandewalle, and Johan Meyers. A parallel-in-time multiple shooting algorithm for large-scale PDE-constrained optimal control problems. *Journal of Computational Physics*, 452:110926, 2022.

[29] Divya Garg, Michael Patterson, William Hager, Anil Rao, David R Benson, and Geoffrey T Huntington. An overview of three pseudospectral methods for the numerical solution of optimal control problems.

[30] Divya Garg, Michael Patterson, William W. Hager, Anil V. Rao, David A. Benson, and Geoffrey T. Huntington. A unified framework for the numerical solution of optimal control problems using pseudospectral methods. *Automatica*, 46(11):1843–1851, November 2010.

[31] Nick Gould, Dominique Orban, and Philippe Toint. CUTEst: A Constrained and Unconstrained Testing Environment with safe threads for Mathematical Optimization. *Computational Optimization and Applications*, 60, July 2014.

[32] Justin S. Gray, John T. Hwang, Joaquim R. R. A. Martins, Kenneth T. Moore, and Bret A. Naylor. OpenMDAO: An open-source framework for multidisciplinary design, analysis, and optimization. *Structural and Multidisciplinary Optimization*, 59(4):1075–1104, April 2019.

[33] Sebastien Gros and Moritz Diehl. Numerical Optimal Control (Draft).

[34] A. Harten, B. Engquist, S. Osher, and S. R. Chakravarthy. Uniformly high order accurate essentially non-oscillatory schemes, III. *Journal of Computational Physics*, 131(1):3–47, 1997.

[35] Himanshu Hatwal and E. C. Mikulcik. Some inverse solutions to an automobile path-tracking problem with input control of steering and brakes. *Vehicle System Dynamics*, 15:61–71, 1986.

[36] Pacejka H.B. *Tyre and Vehicle Dynamics*. Butterworth-Heinemann, 2nd ed edition, 2006.

[37] Alexander Heilmeier, Maximilian Geisslinger, and Johannes Betz. A quasi-steady-state lap time simulation for electrified race cars. In *2019 Fourteenth International Conference on Ecological Vehicles and Renewable Energies (EVER)*. IEEE, May 2019.

[38] HSL. A collection of Fortran codes for large scale scientific computation. http://www.hsl.rl.ac.uk/, 2011.

[39] IPG Automotive. CarMaker. https://www.ipg-automotive.com/solutions/product-portfolio/carmaker, 2026. Accessed: 2026-05-18.

[40] Shuang Li∗ Jisong Zhao. Adaptive mesh refinement method for solving optimal control problems using interpolation error analysis and improved data compression.

[41] Joris Gillis. IPOPT on steroids: Fatrop solver in CasADi, June 2024.

[42] Julia Discourse. 90% of IPOPT run time is spent in Hessian-constraints evaluation, very little time in linear solver — is it normal? https://discourse.julialang.org/t/90-of-ipopt-run-time-is-spent-in-hessian-constraints-evaluation-very-little-ti 136708, 2025. Accessed: 2026-04-19.

[43] JuMP Developers. Performance problem with deeply nested nonlinear expressions. https://github.com/jump-dev/JuMP.jl/issues/4024, 2026. GitHub issue #4024, Accessed: 2026-05-13.

[44] Matthew Kelly. Matthew kelly — personal website. https://www.matthewpeterkelly.com/index.html. Accessed: 2026-05-15.

[45] Matthew Kelly. Trajectory optimization: cannon example. https://www.matthewpeterkelly.com/tutorials/trajectoryOptimization/cannon.html. Accessed: 2026-05-15.

[46] Matthew Kelly. Introduction to trajectory optimization. YouTube, https://www.youtube.com/watch?v=wlkRYMVUZTs, 2017. Accessed: 2026-05-15.

[47] Matthew Kelly. An introduction to trajectory optimization: How to do your own direct collocation. *SIAM Review*, 59(4):849–904, 2017.

[48] Matthew Kelly. An Introduction to Trajectory Optimization: How to Do Your Own Direct Collocation. *SIAM Review*, 59(4):849–904, January 2017.

[49] Matthew Kelly. Trajectory optimization terminology. https://www.matthewpeterkelly.com/tutorials/trajectoryOptimization/terminology.html, 2026. Accessed: 2026-05-14.

[50] Fengjin Liu, William W. Hager, and Anil V. Rao. Adaptive Mesh Refinement Method for Optimal Control Using Decay Rates of Legendre Polynomial Coefficients. *IEEE Transactions on Control Systems Technology*, 26(4):1475–1483, July 2018.

[51] Miles Lubin, Oscar Dowson, Joaquim Dias Garcia, Joey Huchette, Benoît Legat, and Juan Pablo Vielma. JuMP 1.0: Recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*, 2023.

[52] Juan Manzanero. Juanmanzanero/fastest-lap, April 2026.

[53] Matteo Massaro and David Limebeer. Minimum-lap-time optimisation and simulation. *Vehicle System Dynamics*, 59:1–45, April 2021.

[54] mc12027. Mc12027/OpenLAP-Lap-Time-Simulator, April 2026.

[55] Yukio Nakajima. Advanced tire mechanics. 2019.

[56] NVIDIA Corporation. cuDSS: NVIDIA CUDA Direct Sparse Solver Library Documentation. `https://docs.nvidia.com/cuda/cudss/`, 2026. Accessed: 2026-05-12.

[57] OptimumG. How to calculate a corner's importance. `https://optimumg.com/how-to-calculate-a-corners-importance/`. Accessed: 2026-05-18.

[58] Michael A. Patterson, William W. Hager, and Anil V. Rao. A ph mesh refinement method for optimal control. *Optimal Control Applications and Methods*, 36(4):398–421, 2015.

[59] Michael A. Patterson and Anil V. Rao. GPOPS-II: A MATLAB Software for Solving Multiple-Phase Optimal Control Problems Using hp-Adaptive Gaussian Quadrature Collocation Methods and Sparse Nonlinear Programming. *ACM Transactions on Mathematical Software*, 41(1):1–37, October 2014.

[60] Giacomo Perantoni and David J.N. Limebeer. Optimal control for a Formula One car with variable parameters. *Vehicle System Dynamics*, 52(5):653–678, May 2014.

[61] Jesse A Pietz. Pseudospectral Collocation Methods for the Direct Transcription of Optimal Control Problems.

[62] Anil Rao. A Survey of Numerical Methods for Optimal Control. *Advances in the Astronautical Sciences*, 135, January 2010.

[63] R. D. Roland and C. F. Thelin. Computer simulation of Watkins Glen grand prix circuit performance. Technical Report ZL-5002-K-1, Cornell Aeronautical Laboratory, 1971.

[64] Olaf Schenk, Klaus Gärtner, Wolfgang Fichtner, and A.D. Stricker. Pardiso: A high-performance serial and parallel sparse linear solver in semiconductor device simulation. *Future Generation Computer Systems*, 18:69–78, 03 2000.

[65] Martin Schlegel, Klaus Stockmann, Thomas Binder, and Wolfgang Marquardt. Dynamic optimization using adaptive control vector parameterization. *Computers & Chemical Engineering*, 29(8):1731–1751, July 2005.

[66] Sungho Shin, Mihai Anitescu, and François Pacaud. Accelerating optimal power flow with GPUs: SIMD abstraction of nonlinear programs and condensed-space interior-point methods. *Electric Power Systems Research*, 236:110651, 2024.

[67] Sungho Shin, Carleton Coffrin, Kaarthik Sundar, and Victor M Zavala. Graph-based modeling and decomposition of energy infrastructures. *IFAC-PapersOnLine*, 54(3):693–698, 2021.

[68] Sungho Shin, François Pacaud, and Mihai Anitescu. Accelerating optimal power flow with GPUs: SIMD abstraction of nonlinear programs and condensed-space interior-point methods, 2023.

[69] Sungho Shin, François Pacaud, and Mihai Anitescu. How to solve on the GPU a JuMP model — MadNLP.jl documentation. `https://madsuite.org/MadNLP.jl/stable/tutorials/gpu/`, 2026. Accessed 2026-05-21.

[70] Lloyd N Trefethen. Approximation Theory and Approximation Practice.

[71] Lloyd N. Trefethen. *Spectral Methods in Matlab*. Software, Environments, Tools. SIAM: Society for Industrial and Applied Mathematics, 2001.

[72] Charlie Vanaret and Sven Leyffer. *Implementing a unified solver for nonlinearly constrained optimization*. November 2025.

[73] Lander Vanroye, Ajay Sathya, Joris De Schutter, and Wilm Decré. FATROP : A Fast Constrained Optimal Control Problem Solver for Robot Trajectory Optimization and Control, July 2023.

[74] Robin Verschueren, Gianluca Frison, Dimitris Kouzoupis, Jonathan Frey, Niels van Duijkeren, Andrea Zanelli, Branimir Novoselnik, Thivaharan Albin, Rien Quirynen, and Moritz Diehl. acados – a modular open-source framework for fast embedded optimal control. *Mathematical Programming Computation*, 2021.

[75] Matthew J Weinstein and Anil V Rao. Algorithm 984: Adigator, a toolbox for the algorithmic differentiation of mathematical functions in matlab using source transformation via operator overloading. *ACM Transactions on Mathematical Software (TOMS)*, 44(2):21, 2017.

[76] Matthew E. Wilhelm and Matthew D. Stuber. EAGO.jl: easy advanced global optimization in Julia. *Optimization Methods and Software*, 37(2):425–450, 2022.

[77] Hendrik Wilka and Jens Lang. Adaptive hp-Polynomial Based Sparse Grid Collocation Algorithms for Piecewise Smooth Functions with Kinks, May 2025.

[78] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, March 2006.